Dashboard de Facturación

Documentación técnica para instalación, mantenimiento, soporte, comprensión del modelo de datos y transferencia del sistema.

Acceso exclusivo para administradores.

Generada desde README.md · Septiembre de 2026

Índice técnico

1. Documentación disponible
2. Estado del proyecto
3. Estructura
4. Requisitos
5. Instalación local
6. Cómo levantar el proyecto
    7. Opción recomendada: todo junto
    8. Solo Flask
    9. Solo Next.js
    10. Scripts disponibles
11. Variables de entorno principales
12. Usuarios y permisos
13. Base de datos, tablas y datos necesarios
    14. Resumen de tablas/colecciones
    15. Mapa de carga y destino
    16. Ficha de colecciones para la base de datos
    17. Características de las bases de Directorio y control
    18. Datos mínimos para iniciar
    19. usuarios
    20. asignaciones_comerciales
    21. facturacion_anio
    22. justificaciones_ajustes
    23. feriados_operativos
    24. matriz_proyecciones
    25. personal_distribucion_horas
    26. matriz_proyecciones_jornadas
    27. matriz_precios
    28. variables_campanias
    29. tarifaciones_campanias
    30. historial_cambios
31. Dependencias entre tablas
32. Módulos funcionales
    33. Dashboard
    34. Cargar datos
    35. Datos maestros
    36. Control
    37. Justificaciones
    38. Comparativo
    39. Matriz
40. Proyectados
    41. Matriz de proyecciones
    42. Calendario operativo
    43. Matriz de precios
    44. Facturación horas
    45. Variable
    46. Control proyecciones
47. Historial y deshacer
48. Importaciones y plantillas
49. Exportaciones
50. Proyectado vs Facturado y normalización de nombres
51. Cálculos principales
    52. Facturación real
    53. Penalizaciones
    54. Redondeo monetario
    55. Proyección PLP
    56. Consolidado proyectado
57. APIs principales

58. Seguridad
59. Flujo recomendado de uso
    60. Carga posterior de un mes anterior
    61. Generación de un nuevo año proyectado
62. Problemas frecuentes
    63. npm no se reconoce
    64. La app usa otro puerto
    65. No veo cambios en templates
    66. Error de permisos con SQLite
    67. No puedo guardar desde el navegador
68. Notas de desarrollo
69. Guía técnica integral para mantenimiento y transferencia
    70. Arquitectura de ejecución
    71. Propiedad y flujo de los datos
    72. Diccionario completo de tablas
    73. Pipeline exacto de Facturación horas
    74. Tratamiento especial PLP
    75. Suma fija
    76. Correspondencia con el Excel de planificación
    77. Conciliación 2026
    78. Contratos de las APIs de Proyectados
    79. Seguridad y permisos para desarrolladores
    80. Compartir en una red local
    81. Backups y recuperación
    82. Guía para extender un módulo
    83. Checklist de verificación después de un cambio
    84. Diagnóstico de desvíos
85. Anexo DDL completo - SQLite
86. Anexo DDL completo - PostgreSQL

Dashboard de Facturación

Aplicación web para controlar facturación, proyecciones, precios, dotaciones, horas operativas y desvíos comerciales. El sistema combina un backend Flask con base SQLite/PostgreSQL y una experiencia web pensada para uso interno de finanzas, operaciones y control de gestión.

La herramienta centraliza datos reales y proyectados para responder, de forma rápida, preguntas como:

- cuánto se facturó y cuánto se esperaba facturar;
- dónde están los desvíos por cliente, campaña, site o tipo de negocio;
- qué horas, precios, dotaciones, feriados, variables o penalizaciones explican el resultado;
- qué cambios se hicieron y cómo deshacerlos desde el historial.

Documentación disponible

Este repositorio separa la documentación según su público:

Documento	Destinatario	Contenido
`README.md`	Desarrollo, soporte y mantenimiento	Arquitectura, instalación, base de datos, APIs, reglas de cálculo y diagnóstico técnico.
`docs/ENTREGA_DESARROLLO.md`	Desarrollo y DBA	Contrato de conexión, migración SQLite/PostgreSQL, paridad, configuración y aceptación.
`docs/base_actual_manifest.json`	Desarrollo y DBA	Cantidad de filas y huella SHA-256 por tabla de la línea base entregada.
`docs/guia_usuario.html`	Usuarios operativos nuevos o habituales	Guía sencilla con índice y recorridos paso a paso para facturación real, proyectados, Facturación horas, PLP, variables, adicionales, importaciones e historial.

Con la aplicación iniciada, la guía funcional se abre desde **Ayuda > Guía de usuario** o desde:

```
http://127.0.0.1:8009/guia-usuario
```

La guía puede descargarse directamente en PDF por cualquier usuario autenticado. La documentación técnica del README también puede descargarse en PDF, pero esa opción y su endpoint están reservados exclusivamente al rol `administrador`.

```
/guia-usuario/descargar                # Cualquier usuario autenticado
/documentacion-tecnica/descargar       # Solo administrador
```

Estado del proyecto

El proyecto tiene dos frentes:

- **Flask**: backend principal, autenticación, base de datos, APIs y pantallas operativas.
- **Next.js**: frontend en migración progresiva para algunas vistas modernas.

En el uso actual, la mayor parte de las funcionalidades operativas se trabajan desde Flask.

Estructura

```
Dashboard/
├─ app/
│  ├─ __init__.py             # Configuración Flask, seguridad, CORS, CSRF y migraciones
simples
│  ├─ models.py               # Modelos SQLAlchemy y reglas de cálculo
│  ├─ routes.py               # Rutas HTML, APIs, importaciones, historial y exportaciones
│  └─ templates/              # Pantallas Flask
├─ instance/
│  └─ facturacion.db          # Base SQLite local por defecto
├─ next-app/                  # Frontend Next.js en migración
├─ scripts/
│  └─ dev.mjs                 # Levanta Flask + Next en desarrollo
├─ run.py                     # Entrada Flask
├─ requirements.txt           # Dependencias Python
├─ package.json               # Scripts npm del proyecto
├─ .env.example               # Variables de entorno sugeridas
└─ README.md
```

Requisitos

- Python 3.11 o superior.
- pip.
- Node.js con npm, si se quiere levantar también Next.js.
- Navegador moderno.

En Windows, si `npm` no aparece en PowerShell, revisar que exista:

```
C:\Program Files\nodejs\npm.cmd
```

Instalación local

Crear y activar entorno virtual:

```
python -m venv venv
```

```
venv\Scripts\activate
```

Instalar dependencias Python:

```
pip install -r requirements.txt
```

Instalar dependencias del frontend Next.js:

```
npm --prefix next-app install
```

Crear `.env` desde el ejemplo si hace falta:

```
copy .env.example .env
```

Cómo levantar el proyecto

Opción recomendada: todo junto

Desde la carpeta raíz del proyecto:

```
npm run dev
```

Esto levanta:

- Flask/API en `http://127.0.0.1:8009`
- Next.js en `http://127.0.0.1:3000`

Si PowerShell no encuentra `npm`, se puede usar:

```
.\dev.ps1
```

Solo Flask

```
python run.py
```

Por defecto Flask levanta en:

```
http://127.0.0.1:8009
```

Para usar otro puerto, por ejemplo `8010`:

```
$env:PORT="8010"
python run.py
```

Solo Next.js

```
npm run next:dev
```

Scripts disponibles

```
npm run dev        # Flask + Next.js
npm run flask:dev  # Solo Flask
npm run next:dev   # Solo Next.js
npm run build      # Build de producción de Next.js
npm run verify:python  # Esquema, seguridad, rutas Flask y DDL SQLite
npm run verify:handoff # Portabilidad, DDL sincronizados y ausencia de rutas locales fijas
npm run verify:data    # Paridad de la SQLite actual contra el manifiesto entregado
npm run start      # Alias de npm run dev
```

Variables de entorno principales

```
SECRET_KEY=clave-secreta-local
APP_ENV=development
APP_DEFAULT_YEAR=2026
PLP_BASE_YEAR=2026
NAME_ALIASES_FILE=config/name_aliases.json
DATABASE_URL=sqlite:///facturacion.db
CORS_ORIGINS=http://127.0.0.1:3000,http://localhost:3000,http://127.0.0.1:8009,http://localho
st:8009
NEXT_PUBLIC_API_BASE=http://127.0.0.1:8009
FLASK_HOST=127.0.0.1
PORT=8009
FLASK_DEBUG=0
```

```
FLASK_RELOAD=0
TEMPLATES_AUTO_RELOAD=0
SESSION_COOKIE_SECURE=0
SESSION_MINUTES=60
MAX_CONTENT_LENGTH=10485760
```

- `APP_ENV`: use `development` localmente y `production` en el ambiente entregado. En producción, la aplicación rechaza una `SECRET_KEY` ausente o de ejemplo.
- `APP_DEFAULT_YEAR`: período seleccionado por defecto en pantallas y APIs; no limita ni elimina datos de otros años.
- `PLP_BASE_YEAR`: año al que pertenece la matriz histórica PLP incluida. Está separado del período operativo para no reinterpretar esos porcentajes como datos de un año nuevo.
- `NAME_ALIASES_FILE`: archivo versionado con alias ortográficos. No define responsables ni campañas; esas asignaciones se administran en Datos Maestros.
- `DATABASE_URL`: conexión SQLAlchemy; permite cambiar SQLite por PostgreSQL sin modificar el código.
- `CORS_ORIGINS`: orígenes autorizados, separados por coma. También alimentan la política `connect-src` de Flask.
- `NEXT_PUBLIC_API_BASE`: URL pública del backend usada por Next para consultas y enlaces hacia Flask.

Para PostgreSQL:

```
DATABASE_URL=postgresql+psycopg://usuario:password@host:5432/facturacion
```

Usuarios y permisos

El sistema tiene autenticación propia y roles definidos en base de datos.

Roles vigentes: `Full`, `Admin`, `RMO_OPS`, `Finanzas`, `Tesorero`, `RMO` y `RMO_WFM`. Los permisos efectivos se completan con el puesto asociado al rol. Las etiquetas antiguas `superusuario`, `administrador` y `usuario` se convierten automáticamente para mantener compatibilidad con instalaciones previas.

Acciones sensibles como edición, eliminación o deshacer cambios requieren sesión activa, token CSRF y permisos suficientes.

Base de datos, tablas y datos necesarios

La aplicación usa SQLAlchemy. En SQLite o PostgreSQL estos datos viven como **tablas**. Si más adelante se migra a una base documental, se pueden pensar como **colecciones** con la misma responsabilidad funcional.

Al iniciar, Flask crea las tablas que falten y aplica algunos ajustes simples de esquema. Aun así, para que la herramienta tenga información útil, hay datos mínimos que conviene cargar en cada módulo.

Resumen de tablas/colecciones

Tabla / colección	Modelo	Para qué sirve
`usuarios`	`Usuario`	Guarda usuarios, roles, estado activo y hash de contraseña.
`historial_cambios`	`HistorialCambio`	Registra cambios, snapshots anteriores/posteriores y permite deshacer operaciones soportadas.
`campanias`	`Campania`	Catálogo canónico que asigna un ID estable a cada combinación cliente + campaña.
`facturacion_anio`	`FacturacionAnio`	Guarda la facturación real histórica y futura desde 2020, sin limitarla a un año fijo.
`justificaciones_ajustes`	`JustificacionAjuste`	Guarda explicaciones o ajustes asociados a un registro de facturación.
`asignaciones_comerciales`	`AsignacionComercial`	Catálogo maestro de cliente, gerente, jefe de site, campaña, subcampaña y tipo de negocio.
`matriz_proyecciones`	`ProyeccionMatriz`	Guarda la planificación mensual por cliente/campaña: dotación, carga horaria, días objetivo, horas requeridas y cumplimiento.
`matriz_proyecciones_jornadas`	`ProyeccionMatrizJornada`	Guarda aperturas de jornadas dentro de una proyección cuando una campaña necesita más de una configuración horaria.
`proforma_personal`	`ProformaPersonal`	Guarda proformas mensuales de Personal por tipo, estado, segmento, tipo de hora, precio y componentes variables.
`seguimiento_personal_importaciones`	`SeguimientoPersonalImportacion`	Conserva los libros importados de seguimiento de Personal, sus hojas, fórmulas, advertencias y usuario responsable.
`personal_distribucion_horas`	`PersonalDistribucionHoras`	Guarda por año, mes y clasificación PLP el porcentaje de horas diurnas; la nocturnidad es la diferencia hasta 100%.
`matriz_precios`	`ProyeccionPrecio`	Guarda precios mensuales por cliente/campaña, alcance, precio final e importe fijo mensual.
`variables_campanias`	`VariableCampania`	Guarda el porcentaje variable mensual por cliente/campaña.
`tarifaciones_campanias`	`TarifacionCampania`	Guarda montos mensuales positivos o negativos que se suman a Facturación horas.
`next_gen_dolar`	`NextGenDolar`	Guarda el valor del dólar utilizado por Next Gen para cada mes y año.

Tabla / colección	Modelo	Para qué sirve
`next_gen_productos`	`NextGenProducto`	Guarda cantidades mensuales en USD por cliente, campaña y producto Next Gen.
`sites_proyecciones`	`SiteProyeccion`	Guarda la regularización manual del site o gerencia de una campaña proyectada.
`feriados_operativos`	`FeriadoOperativo`	Guarda feriados o días no operativos que afectan los días objetivo de las proyecciones.
`dotaciones_mensuales`	`DotacionMensual`	Serie mensual consolidada de dotación requerida y personal para indicadores.
`ratio_eli_mensual`	`RatioEliMensual`	Serie mensual de requerido y staff para el indicador ELI.
`ratio_eli_ii_mensual`	`RatioEliIIMensual`	Serie mensual operativa y monetaria de operaciones y staff para ELI II.
`dashboard_operativo`	`DashboardOperativo`	Base mensual por cliente/campaña de Directorio; conserva datos importados y bajas manuales.
`historico_clientes_mensuales`	`HistoricoClienteMensual`	Base histórica mensual por cliente; conserva datos fuente, pagadas y logueo.
`dotaciones_clientes_mensuales`	`DotacionClienteMensual`	Dotación requerida mensual abierta por cliente.
`graficos_dotaciones_mensuales`	`GraficoDotacionMensual`	Serie mensual para gráficos de dotación requerida y activa por sede.
`excepciones_calculo`	`ExcepcionCalculo`	Reglas excepcionales y ajustes porcentuales por cliente/campaña.
`normalizacion_nombres_proyectados`	`NormalizacionNombreProyectado`	Vincula cada nombre canónico de Facturación con el cliente y servicio equivalente de Proyectados.

Mapa de carga y destino

Tabla	Dónde se carga	A dónde apunta / qué alimenta
`usuarios`	Usuarios o configuración inicial	Login, sesiones, permisos e identificación del historial.
`campanias`	Automática al crear o migrar una asignación	ID canónico utilizado por asignaciones e integraciones nuevas.
`asignaciones_comerciales`	Datos maestros	Formularios, filtros, sites, clientes y campañas de todos los módulos.

Tabla	Dónde se carga	A dónde apunta / qué alimenta
`facturacion_anio`	Cargar datos, importación de facturación o edición en Control	Dashboard, Control, Comparativo, Matriz, Justificaciones y exportaciones reales.
`justificaciones_ajustes`	Justificaciones	Un registro concreto de `facturacion_anio`; ajusta y explica bonos, penalizaciones u otros conceptos.
`feriados_operativos`	Calendario operativo	Días objetivo, horas y dotaciones de `matriz_proyecciones`.
`matriz_proyecciones`	Matriz de proyecciones, importación general o importación PLP	Control proyecciones y base de horas de Facturación horas.
`matriz_proyecciones_jornadas`	Aperturas de jornada dentro de una proyección	Suma de horas y dotaciones de su registro padre en `matriz_proyecciones`.
`proforma_personal`	Facturación > Carga de datos Personal > Proforma	Alimenta el seguimiento de Personal y permite publicar períodos definitivos en el Dashboard.
`seguimiento_personal_importaciones`	Facturación > Carga de datos Personal > Seguimiento	Conserva el archivo fuente y permite revisar, completar, cerrar y descargar el seguimiento procesado.
`personal_distribucion_horas`	Matriz de proyecciones > PLP > porcentajes	Apertura diurna/nocturna de las proyecciones Personal y sus cálculos posteriores.
`matriz_precios`	Matriz de precios o importación anual	Valoriza horas proyectadas, nocturnidad e importes fijos en Facturación horas.
`variables_campanias`	Variable > % Variable o importación anual	Estimación variable y total consolidado de Facturación horas.
`tarifaciones_campanias`	Variable > Tarifación o importación anual	Ajuste monetario sumado o restado en Facturación horas.
`next_gen_dolar`	Variable > Next Gen o importación anual	Cotización mensual usada para convertir productos Next Gen a pesos.
`next_gen_productos`	Variable > Next Gen o importación anual	Importe Next Gen en pesos y total consolidado de Facturación horas.
`sites_proyecciones`	Edición directa de la columna Site en Facturación horas	Reagrupa todos los meses y conceptos consolidados de la campaña.
`historial_cambios`	Automática; no se carga manualmente	Auditoría y restauración de snapshots en operaciones compatibles con Deshacer.
`dotaciones_mensuales`	Importación de Directorio / Dotaciones	Indicadores consolidados de requerida, personal y diferencia mensual.

Tabla	Dónde se carga	A dónde apunta / qué alimenta
`ratio_eli_mensual`	Importación del indicador ELI	Gráfico mensual de requerido, staff y ratio calculado.
`ratio_eli_ii_mensual`	Importación del indicador ELI II	Gráficos operativos y monetarios; los totales y ratios se calculan al consultar.
`dashboard_operativo`	Importación Dashboard operativo; edición de bajas	Tableros de Directorio por año, mes, cliente y campaña.
`historico_clientes_mensuales`	Importación histórica; edición de pagadas/logueo	Histórico y comparativos mensuales por cliente.
`dotaciones_clientes_mensuales`	Importación de dotaciones por cliente	Cuadros, evolutivos y comparaciones de dotación requerida.
`graficos_dotaciones_mensuales`	Importación de gráficos de dotaciones	Series de requerida, activa SL, activa BA, diferencia y sobre-staff.
`excepciones_calculo`	Configuración técnica	Ajustes puntuales del cálculo sin alterar la fórmula general.
`normalizacion_nombres_proyectados`	Datos maestros > Normalizar nombres proyectados	Unifica las denominaciones usadas por Proyectados y Facturación en el análisis ejecutivo y sus exportaciones.

Ficha de colecciones para la base de datos

Esta sección resume qué necesita cada tabla/colección para que la base quede armada correctamente.

Contrato de precisión numérica

Los campos numéricos persistentes no utilizan `FLOAT`. La base declara precisión y escala explícitas para evitar diferencias binarias, especialmente en dinero.

Categoría	Tipo SQL	Alcance
Dinero	`NUMERIC(18,2)`	Hasta 16 enteros y 2 decimales
Cantidades	`NUMERIC(18,4)`	Hasta 14 enteros y 4 decimales
Horas y dotación	`NUMERIC(12,2)`	Hasta 10 enteros y 2 decimales
Porcentajes	`NUMERIC(9,4)`	Hasta 5 enteros y 4 decimales
Cotización monetaria	`NUMERIC(18,6)`	Hasta 12 enteros y 6 decimales

Aplicación por campo:

```
NUMERIC(18,2) Dinero
  facturacion_anio.valor_hora_objetivo
  facturacion_anio.valor_hora
  facturacion_anio.tarifacion
  facturacion_anio.importe_fijo
  facturacion_anio.variable_objetivo
  facturacion_anio.variable_productivo
  facturacion_anio.bonos
  facturacion_anio.penalizaciones
  facturacion_anio.netx_gen
  facturacion_anio.otros
  justificaciones_ajustes.precio
  justificaciones_ajustes.importe
  matriz_precios.precio_base
  matriz_precios.precio_final
  matriz_precios.importe_fijo_mensual
  tarifaciones_campanias.monto
  next_gen_productos.cantidad_usd

NUMERIC(18,4) Cantidades
  justificaciones_ajustes.cantidad

NUMERIC(12,2) Horas y dotación
  facturacion_anio.horas_objetivo
  facturacion_anio.horas_facturadas
  facturacion_anio.horas_penalizadas
  matriz_proyecciones.dotacion_requerida
  matriz_proyecciones.carga_horaria
  matriz_proyecciones.horas_requeridas
  matriz_proyecciones.horas_requeridas_manual
  matriz_proyecciones_jornadas.dotacion_requerida
  matriz_proyecciones_jornadas.carga_horaria
  matriz_proyecciones_jornadas.horas_requeridas

NUMERIC(9,4) Porcentajes
  matriz_proyecciones.porcentaje_cumplimiento
  matriz_proyecciones.porcentaje_nocturnidad
  personal_distribucion_horas.porcentaje_diurno
  matriz_precios.alcance_porcentaje
  variables_campanias.porcentaje

NUMERIC(18,6) Cotización
  next_gen_dolar.valor
```

En PostgreSQL, el arranque convierte columnas históricas con `ALTER COLUMN ... TYPE NUMERIC(...)`. En SQLite, los DDL nuevos declaran `NUMERIC`, aunque SQLite mantiene afinidad dinámica y no aplica precisión/escala con la misma rigidez que PostgreSQL. Para la base definitiva creada por un tercero, el contrato autoritativo es el DDL PostgreSQL.

`campanias`

Modelo: `Campania`.

Es el catálogo canónico de campañas. Cada fila tiene una clave primaria numérica autoincremental en `campanias.id`. La identidad funcional se define por la combinación `cliente + nombre`, no solamente por el texto del nombre.

```
campanias.id = ID técnico estable
campanias.cliente + campanias.nombre = identidad funcional única
```

Cómo se genera la numeración

La numeración no debe ingresarla el usuario. La genera automáticamente la base de datos cuando se crea por primera vez una combinación `cliente + campaña`:

```
Primera campaña nueva  -> id 1
Segunda campaña nueva  -> id 2
Tercera campaña nueva  -> id 3
Siguiente campaña      -> id 4
```

Si una campaña ya existe, se reutiliza su ID y no se consume otro número. Si el último ID existente fuera `92`, la próxima campaña nueva recibiría normalmente el ID `93`. Los IDs son identificadores técnicos: pueden quedar saltos si una fila se elimina y no deben renumerarse ni reutilizarse para cerrar esos saltos.

Al incorporar esta estructura a una base local que ya tiene datos, `ensure_schema()` enumera las campañas históricas existentes, crea el catálogo y completa `asignaciones_comerciales.campania_id`. Por lo tanto, no hace falta volver a crear manualmente las campañas que ya estaban cargadas.

id	cliente	nombre	activa
1	Cliente A	Ventas	true
2	Cliente A	Retención	true
3	Cliente B	Ventas	true

Los registros 1 y 3 pueden compartir `Ventas` porque pertenecen a clientes distintos. La restricción `uq_campanias_cliente_nombre` impide duplicar el mismo nombre dentro del mismo cliente.

`asignaciones_comerciales.campania_id` referencia este ID. La columna textual `asignaciones_comerciales.campania` se conserva durante la transición para no romper filtros, importadores, históricos ni módulos que todavía trabajan por nombre. Las integraciones nuevas deben usar `campania_id` como clave técnica.

Reglas de creación y mantenimiento:

1. al crear una asignación se busca el mismo cliente y nombre en `campanias`;
2. si existe, se reutiliza su ID y la campaña se reactiva;

3. si no existe, se inserta una fila y la base genera el siguiente ID;
4. si cambia cliente o campaña en una asignación, se vuelve a resolver el ID;
5. `ensure_schema()` crea campañas faltantes y completa el ID de asignaciones históricas;
6. el ID nunca debe calcularse desde el nombre ni asignarse manualmente.

Ejemplo del contrato JSON del catálogo:

```
{
  "id": 1,
  "cliente": "Cliente A",
  "nombre": "Ventas",
  "activa": true,
  "creado_en": "2026-07-22T12:00:00"
}
```

Durante la transición, las asignaciones publican el ID y el nombre:

```
{
  "id": 25,
  "campania_id": 1,
  "cliente": "Cliente A",
  "campania": "Ventas"
}
```

El ID también es visible en la pantalla **Datos maestros**, en la columna `ID campaña`. Ese valor corresponde a `campanias.id`; no debe confundirse con el ID propio de la asignación comercial utilizado para editar la fila.

API de consulta:

```
GET /api/campanias            # Sólo activas
GET /api/campanias?activas=0  # Activas e inactivas
```

Ambas variantes requieren una sesión con permiso de edición.

Especificación técnica para crear y revisar la base

Esta es la definición que debe utilizar el responsable de base de datos. Para el destino PostgreSQL, `campanias.id` es una identidad generada por la base y `asignaciones_comerciales.campania_id` es su clave foránea.

DDL recomendado para PostgreSQL:

```
CREATE TABLE campanias (
    id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    cliente VARCHAR(100) NOT NULL,
    nombre VARCHAR(100) NOT NULL,
```

```
    activa BOOLEAN NOT NULL DEFAULT TRUE,
    creado_en TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_campanias_cliente_nombre UNIQUE (cliente, nombre)
);

ALTER TABLE asignaciones_comerciales
    ADD COLUMN campania_id INTEGER;

ALTER TABLE asignaciones_comerciales
    ADD CONSTRAINT fk_asignaciones_campania
    FOREIGN KEY (campania_id) REFERENCES campanias(id);

CREATE INDEX ix_asignaciones_comerciales_campania_id
    ON asignaciones_comerciales (campania_id);
```

Para una base nueva creada directamente por SQLAlchemy, el modelo de `app/models.py` genera la tabla, el índice y la clave foránea. Para una base con datos históricos, primero se crea y completa el catálogo, después se valida que no existan referencias nulas o inválidas y recién entonces se recomienda cambiar `campania_id` a `NOT NULL`:

```
ALTER TABLE asignaciones_comerciales
    ALTER COLUMN campania_id SET NOT NULL;
```

Definición equivalente del catálogo para SQLite local:

```
CREATE TABLE campanias (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    cliente VARCHAR(100) NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    activa BOOLEAN NOT NULL DEFAULT 1,
    creado_en DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_campanias_cliente_nombre UNIQUE (cliente, nombre)
);

ALTER TABLE asignaciones_comerciales ADD COLUMN campania_id INTEGER;

CREATE INDEX ix_asignaciones_comerciales_campania_id
    ON asignaciones_comerciales (campania_id);
```

> Importante: SQLite no permite agregar una clave foránea física a una tabla existente con un `ALTER TABLE` simple. En la base local histórica la relación se completa y controla desde la aplicación. Una base nueva sí recibe la clave foránea declarada por SQLAlchemy. PostgreSQL debe tener la restricción física `fk_asignaciones_campania` indicada arriba.

Consultas obligatorias de revisión:

```
-- 1. No debe haber cliente + nombre duplicados.
SELECT cliente, nombre, COUNT(*) AS cantidad
FROM campanias
GROUP BY cliente, nombre
HAVING COUNT(*) > 1;

-- 2. No debe haber asignaciones sin ID de campaña.
SELECT COUNT(*) AS asignaciones_sin_campania_id
FROM asignaciones_comerciales
WHERE campania_id IS NULL;

-- 3. No debe haber IDs que apunten a una campaña inexistente.
SELECT a.id, a.campania_id
FROM asignaciones_comerciales a
LEFT JOIN campanias c ON c.id = a.campania_id
WHERE a.campania_id IS NOT NULL AND c.id IS NULL;

-- 4. El ID debe corresponder al mismo cliente y nombre conservados en la asignación.
SELECT a.id, a.cliente, a.campania, a.campania_id,
       c.cliente AS cliente_catalogo, c.nombre AS campania_catalogo
FROM asignaciones_comerciales a
JOIN campanias c ON c.id = a.campania_id
WHERE a.cliente <> c.cliente OR a.campania <> c.nombre;
```

Criterio de aceptación para aprobar la base:

```
[ ] campanias.id es clave primaria y se genera automáticamente.
[ ] Existe unicidad para cliente + nombre.
[ ] asignaciones_comerciales.campania_id está indexado.
[ ] En PostgreSQL existe la clave foránea fk_asignaciones_campania.
[ ] Las cuatro consultas de revisión no devuelven anomalías.
[ ] Crear la misma combinación cliente + campaña reutiliza el mismo ID.
[ ] Crear una combinación nueva genera un ID nuevo.
[ ] Datos maestros muestra el mismo valor en la columna ID campaña.
```

Entregable completo de creación para terceros

El tercero responsable de crear o revisar la base debe recibir uno de estos archivos, según el motor acordado:

Motor	Archivo contractual	Descarga desde la aplicación
SQLite local	`docs/esquema_base_datos_sqlite.sql`	`/documentacion-tecnica/esquema/sqlite`
PostgreSQL	`docs/esquema_base_datos_postgresql.sql`	`/documentacion-tecnica/esquema/postgresql`

Quien necesite revisar el esquema sin abrir archivos `.sql` puede descargar el PDF asociado a la documentación técnica:

```
/documentacion-tecnica/esquema/pdf
```

El archivo entregado es `Esquema_Base_Datos_Dashboard.pdf` e incluye juntos los DDL completos de SQLite y PostgreSQL.

Los scripts contienen, tabla por tabla, la definición física en formato `CREATE TABLE` y `CREATE INDEX`, tal como se entrega a un DBA:

- nombre exacto de cada tabla;
- todas las columnas y sus tipos;
- longitud de los `VARCHAR`;
- columnas obligatorias mediante `NOT NULL`;
- valores `DEFAULT` físicos, cuando estén definidos en el servidor;
- claves primarias;
- claves foráneas y tablas referenciadas;
- restricciones `UNIQUE`;
- índices y columnas indexadas.

El esquema incluye las 27 tablas vigentes:

```
usuarios
historial_cambios
facturacion_anio
justificaciones_ajustes
matriz_proyecciones
matriz_proyecciones_jornadas
personal_distribucion_horas
matriz_precios
variables_campanias
tarifaciones_campanias
next_gen_dolar
next_gen_productos
sites_proyecciones
feriados_operativos
dotaciones_mensuales
ratio_eli_mensual
ratio_eli_ii_mensual
dashboard_operativo
historico_clientes_mensuales
dotaciones_clientes_mensuales
graficos_dotaciones_mensuales
campanias
excepciones_calculo
asignaciones_comerciales
normalizacion_nombres_proyectados
proformas_personal
seguimiento_personal_importaciones
```

Características de las bases de Directorio y control

Tabla	Granularidad / clave lógica	Datos persistidos	Cálculos derivados y reglas
`dotaciones_mensuales`	Una fila por `fecha` mensual, única	Dotación requerida y personal	Diferencia y porcentajes se calculan al consultar; no se duplican meses.
`ratio_eli_mensual`	Una fila por `fecha` mensual, única	Requerido y staff	`ratio = staff / requerido`; si requerido es cero, el ratio queda sin valor.
`ratio_eli_ii_mensual`	Una fila por `fecha` mensual, única	Operaciones, staff e importes de ambos	Total y ratios operativos/monetarios se derivan de los cuatro valores base.
`dashboard_operativo`	`mes + cliente + campaña`, único	Año, datos importados en JSON y bajas manuales	La reimportación actualiza la misma clave y conserva el tratamiento explícito de bajas.
`historico_clientes_mensuales`	`mes + cliente`, único	Base fuente en JSON, pagadas y logueo	Pagadas y logueo son editables; la información importada permanece agrupada por cliente/mes.
`dotaciones_clientes_mensuales`	`cliente + fecha`, único	Dotación requerida	Alimenta cuadros y evolutivos abiertos por cliente sin duplicar el mismo período.
`graficos_dotaciones_mensuales`	Una fila por `fecha` mensual, única	Requerida, activa SL y activa BA	Activa total, diferencia y porcentaje de sobre-staff se calculan al consultar.
`excepciones_calculo`	`cliente + campaña`, único	Tipo de cálculo, ajustes de valor hora objetivo/alcanzado y estado	Solo se aplica cuando está activa; permite excepciones trazables sin forzar los datos generales.

Los campos JSON de `dashboard_operativo.datos` e `historico_clientes_mensuales.datos_base` conservan las columnas variables de la fuente importada. Los campos que participan en filtros, edición o unicidad se mantienen como columnas tipadas. El detalle físico de **cada campo**, tipo, obligatoriedad, clave, índice y restricción está en los anexos DDL generados que acompañan esta ficha.

Estos archivos se generan directamente desde `app/models.py`; por eso son la fuente técnica para construir una base nueva y evitan copiar accidentalmente datos reales de `instance/facturacion.db`. Cada cambio de modelos exige volver a generar los scripts y la documentación:

Compatibilidad con bases locales anteriores

El contrato nuevo está formado por las **27 tablas declaradas en los DDL**. Una base SQLite local actualizada durante versiones anteriores puede conservar la tabla vacía `bajas_operativas`, las columnas vacías

`facturacion_anio.mes_baja` y `facturacion_anio.motivo_baja`, o las columnas en cero `matriz_precios.aplica_dolar` y `matriz_precios.valor_dolar`. Son artefactos legacy no consultados por la aplicación y **no deben crearse en una base nueva**. La valorización vigente en dólares utiliza `next_gen_dolar`, `next_gen_productos.cantidad_usd` y `next_gen_productos.cotizacion_aplicada`.

> Nota: varios valores predeterminados actuales son aplicados por SQLAlchemy desde Python y no son `DEFAULT` físicos del motor. El DDL muestra deliberadamente sólo las garantías que la base puede imponer por sí misma; esto permite que el tercero distinga las restricciones de base de las reglas ejecutadas por la aplicación.

```
python scripts\generar_esquema_base_datos.py
python scripts\generar_documentacion_tecnica.py
```

Para PostgreSQL debe utilizarse el archivo PostgreSQL, no convertir manualmente el de SQLite: autoincrementos, booleanos, fechas y tipos se expresan de manera diferente en cada motor.

`usuarios`

- **Propósito**: administrar acceso, roles y estado de los usuarios.
- **Campos clave**: `nombre`, `email`, `password_hash`, `rol`, `activo`.
- **Obligatorio para iniciar**: sí. Debe existir al menos un usuario administrador.
- **Reglas importantes**: el email debe ser único; la contraseña se guarda como hash; los roles habilitan o bloquean acciones sensibles.
- **Carga manual recomendada**: solo usuarios reales o de administración.

`asignaciones_comerciales`

- **Propósito**: catálogo maestro que define combinaciones válidas de cliente, gerente/site, jefe, campaña, subcampaña y tipo de negocio.
- **Campos clave**: `cliente`, `gerente`, `jefe_site`, `campania`, `subcampania`, `tipo_negocio`, `activa`.
- **Obligatorio para iniciar**: sí, antes de proyecciones, precios y variables.
- **Reglas importantes**: cliente y campaña deben escribirse igual que en facturación, proyecciones y precios; una asociación inactiva no debería usarse para cargas nuevas.
- **Uso histórico**: si cambia el jefe desde un mes puntual, usar **Aplicar desde mes** para no pisar meses anteriores.

`facturacion_anio`

- **Propósito**: guardar la facturación real cargada por mes y dimensión comercial.
- **Cobertura temporal**: admite información desde 2020 en adelante y no está vinculada a un año fijo.
- **Campos clave**: `fecha`, `mes`, `cliente`, `gerente`, `jefe_site`, `campania`, `subcampania`, `tipo_negocio`, `tipo_jornada`, horas e importes.

- **Obligatorio para iniciar**: sí, si se quiere usar Dashboard, Control, Comparativo, Matriz, Justificaciones y exportaciones.
- **Reglas importantes**: `mes` debe estar en formato `YYYY-MM`; las dimensiones comerciales deben coincidir con `asignaciones_comerciales`; penalizaciones se tratan como descuento.
- **Datos mínimos útiles**: fecha, mes, cliente, gerente, jefe, campaña, subcampaña, tipo de jornada, horas objetivo, horas facturadas y valor hora.

`justificaciones_ajustes`

- **Propósito**: explicar ajustes, bonos, penalizaciones u otros conceptos asociados a una carga real.
- **Campos clave**: `facturacion_id`, `tipo`, `cantidad`, `precio`, `importe`, `descripcion`.
- **Obligatorio para iniciar**: no.
- **Reglas importantes**: cada justificación debe apuntar a un registro existente de `facturacion_anio`; en penalizaciones, el importe impacta como descuento.
- **Carga manual recomendada**: solo cuando un desvío o ajuste necesite respaldo.

`matriz_proyecciones`

- **Propósito**: guardar la planificación mensual de horas y dotaciones por cliente/campaña.
- **Campos clave**: `cliente`, `campania`, `year`, `mes`, `dotacion_requerida`, `carga_semanal`, `carga_horaria`, `dias_objetivo`, `horas_requeridas`, `porcentaje_cumplimiento`.
- **Obligatorio para iniciar**: sí, si se usan Proyectados, Facturación horas, Variable o Control proyecciones.
- **Clave funcional**: `cliente + campania + mes`.
- **Reglas importantes**: `mes` debe estar en formato `YYYY-MM`; cliente/campaña deben existir en Datos maestros; feriados activos pueden recalcular días objetivo.

`matriz_proyecciones_jornadas`

- **Propósito**: abrir una proyección en varias jornadas cuando una campaña tiene más de una configuración horaria dentro del mismo mes.
- **Campos clave**: `proyeccion_id`, `dotacion_requerida`, `carga_semanal`, `carga_horaria`, `dias_objetivo`, `horas_requeridas`.
- **Obligatorio para iniciar**: no.
- **Reglas importantes**: cada fila depende de una proyección existente en `matriz_proyecciones`.
- **Cuándo usarla**: solo cuando una misma campaña/mes necesita separar jornadas.

`personal_distribucion_horas`

- **Propósito**: definir la apertura diurna/nocturna de las proyecciones PLP de Personal.
- **Campos clave**: `servicio`, `year`, `mes`, `porcentaje_diurno`.
- **Clasificaciones admitidas**: `Personal CX`, `Personal`, `Personal Soporte` y `Personal SMB`.
- **Clave funcional**: `servicio + mes`.

- **Regla principal**: `porcentaje_nocturno = 100 - porcentaje_diurno`.
- **Destino**: recalcula las aperturas principal y `nocturnidad` de `matriz_proyecciones`; ambas alimentan Matriz de precios, Facturación horas, Variable y Control proyecciones.
- **Cambio de año**: si el año seleccionado no tiene valores propios, la pantalla propone los porcentajes del año anterior. Al guardarlos se crean registros para el nuevo año sin modificar el histórico.
- **Carga manual recomendada**: revisar los doce porcentajes de cada clasificación antes de importar o recalcular PLP.

`matriz_precios`

- **Propósito**: guardar precios mensuales para calcular facturación proyectada.
- **Campos clave**: `site`, `cliente`, `campania`, `year`, `mes`, `precio_base`, `alcance_porcentaje`, `precio_final`, `importe_fijo_mensual`.
- **Obligatorio para iniciar**: sí, si se quiere calcular Facturación horas.
- **Clave funcional**: `cliente + campania + mes`.
- **Reglas importantes**: debe coincidir con `matriz_proyecciones` en cliente/campaña/mes; `alcance_porcentaje` normalmente es `100`; `importe_fijo_mensual` puede quedar en `0`.

`variables_campanias`

- **Propósito**: guardar el porcentaje variable mensual aplicado sobre Facturación horas.
- **Campos clave**: `site`, `cliente`, `campania`, `year`, `mes`, `porcentaje`.
- **Obligatorio para iniciar**: no, salvo que se necesite calcular variables.
- **Clave funcional**: `cliente + campania + mes`.
- **Reglas importantes**: la campaña debe existir en Facturación horas; el porcentaje puede ser `0`; los cambios quedan auditados y pueden deshacerse desde Historial cuando aplica.

`tarifaciones_campanias`

- **Propósito**: guardar ajustes monetarios independientes de las horas y del porcentaje variable.
- **Campos clave**: `site`, `cliente`, `campania`, `concepto`, `year`, `mes`, `monto`.
- **Clave funcional**: `cliente + campania + concepto + mes`.
- **Reglas importantes**: el monto conserva su signo; un valor positivo suma y uno negativo resta. La coincidencia con Facturación horas ignora mayúsculas, minúsculas y acentos.
- **Destino**: se incorpora al mismo renglón consolidado de la campaña en Facturación horas.
- **Carga**: se puede ingresar manualmente o importar todo el año desde la subpestaña **Tarifación**.
- **Auditoría**: las creaciones, ediciones e importaciones quedan identificadas como Tarifación en Historial y se pueden deshacer.

`feriados_operativos`

- **Propósito**: registrar días no operativos que impactan en los días objetivo de proyecciones.

- **Campos clave**: `year`, `fecha`, `nombre`, `tipo`, `activo`.
- **Obligatorio para iniciar**: no, salvo que las proyecciones deban descontar feriados.
- **Clave funcional**: `fecha`.
- **Reglas importantes**: solo feriados activos impactan cálculos; al agregar o eliminar un feriado activo, se recalculan proyecciones del mes afectado.

`historial_cambios`

- **Propósito**: auditar operaciones relevantes y guardar snapshots para trazabilidad.
- **Campos clave**: `usuario_id`, `usuario_nombre`, `usuario_email`, `accion`, `entidad`, `entidad_id`, `resumen`, `detalle`, `antes`, `despues`.
- **Obligatorio para iniciar**: no se carga manualmente.
- **Reglas importantes**: la aplicación lo completa automáticamente; permite revisar cambios y deshacer algunas operaciones soportadas.
- **Uso esperado**: auditoría de creación, edición, eliminación, importaciones, inflación, precios, variables y proyecciones.

Datos mínimos para iniciar

Para una base nueva, el orden recomendado de carga es:

1. `usuarios`: crear al menos un usuario `Admin` o `Full`.
2. `asignaciones_comerciales`: cargar el maestro comercial base.
3. `facturacion_anio`: cargar registros reales si se quiere usar Dashboard, Control, Matriz, Comparativo y Justificaciones.
4. `feriados_operativos`: cargar feriados si las proyecciones deben descontar días no operativos.
5. `matriz_proyecciones`: cargar horas y dotaciones proyectadas por mes.
6. `personal_distribucion_horas`: configurar los porcentajes diurnos/nocturnos de PLP para el año correspondiente.
7. `matriz_precios`: cargar precios por mes para cruzar contra las horas proyectadas, incluyendo los nombres de nocturnidad cuando tengan un precio específico.
8. `variables_campanias`: cargar porcentajes variables una vez que exista Facturación horas.
9. `tarifaciones_campanias`: cargar ajustes monetarios mensuales positivos o negativos cuando correspondan.

`usuarios`

Campos principales:

Campo	Descripción
`nombre`	Nombre visible del usuario.
`email`	Email único para iniciar sesión.
`password_hash`	Contraseña hasheada, nunca texto plano.
`rol`	Rol del usuario. Valores vigentes: `Full`, `Admin`, `RMO_OPS`, `Finanzas`, `Tesorero`, `RMO` y `RMO_WFM`.
`activo`	Define si puede iniciar sesión.

Datos que necesitás agregar:

- al menos un usuario `Admin` o `Full`;
- usuarios operativos según quién cargue o consulte información.

`asignaciones_comerciales`

Es la tabla maestra más importante para ordenar la aplicación.

Campos principales:

Campo	Descripción
`cliente`	Cliente comercial.
`gerente`	Gerencia o site principal usado para agrupación.
`jefe_site`	Responsable o jefe de site.
`campania`	Campaña, con soporte para acentos y Ñ.
`subcampania`	Subcampaña o apertura secundaria.
`tipo_negocio`	Clasificación operativa/comercial.
`activa`	Indica si aparece en formularios y filtros.

Datos que necesitás agregar:

- una fila por combinación comercial válida;
- cliente y campaña exactamente como se van a usar luego en facturación, proyecciones y precios;
- site/gerente correcto para que los filtros agrupen bien.

Esta tabla alimenta formularios de carga, filtros, Matriz de proyecciones, Matriz de precios, Facturación horas, Variable y Control proyecciones.

Cambios de jefe de site con vigencia mensual

Cuando una campaña cambia de responsable a partir de un mes determinado, no se debe pisar toda la asociación histórica. En Datos maestros se puede editar la asociación y completar **Aplicar desde mes** con formato `YYYY-MM`.

Ejemplo:

- `Santander Chat` estuvo bajo la jefatura de `Ismael Rissi` hasta `2026-05`.
- Desde `2026-06` pasa a `Mariela Ditto`.

Al editar la asociación y cargar `2026-06` en **Aplicar desde mes**, el sistema:

- mantiene los registros anteriores a junio con `Ismael Rissi`;
- actualiza desde junio en adelante con `Mariela Ditto`;
- crea o activa una nueva asociación para la combinación vigente;
- registra la operación en Historial.

Si **Aplicar desde mes** queda vacío, la edición se comporta como una actualización general de la asociación y de sus cargas vinculadas.

`facturacion_anio`

Guarda la facturación real.

Campos principales:

Campo	Descripción
`fecha`	Fecha del registro.
`mes`	Mes de análisis.
`cliente`, `gerente`, `jefe_site`, `campania`, `subcampania`, `tipo_negocio`	Dimensiones comerciales.
`tipo_jornada`	Tipo de jornada o tipo VH.
`horas_objetivo`	Horas esperadas.
`horas_facturadas`	Horas efectivamente facturadas.
`horas_penalizadas`	Horas descontadas por penalización.
`valor_hora_objetivo`	Valor hora esperado.

Campo	Descripción
`valor_hora`	Valor hora alcanzado/facturado.
`tarifacion`	Ajuste adicional de tarifación.
`importe_fijo`	Importe fijo si no aplica cálculo por horas.
`variable_objetivo`	Bono/variable objetivo.
`variable_productivo`	Variable productiva real.
`bonos`	Bonos facturados.
`penalizaciones`	Penalizaciones, tratadas como descuento.
`netx_gen` , `otros`	Otros importes de apertura.

Datos que necesitás agregar:

- registros reales por mes;
- horas e importes necesarios para calcular objetivo, real, desvío y cumplimiento;
- dimensiones comerciales consistentes con `asignaciones_comerciales` .

Esta tabla alimenta Dashboard, Control, Comparativo, Matriz, Justificaciones y exportaciones.

`justificaciones_ajustes`

Guarda explicaciones o ajustes vinculados a un registro de `facturacion_anio` .

Campos principales:

Campo	Descripción
`facturacion_id`	Registro de facturación asociado.
`tipo`	Tipo de ajuste, por ejemplo penalización, bono u otro concepto.
`cantidad`	Cantidad del ajuste.
`precio`	Precio unitario.
`importe`	Importe total.
`descripcion`	Explicación del ajuste.

Datos que necesitás agregar:

- solo cuando un registro necesita explicación, ajuste o soporte documental;
- en penalizaciones, el sistema interpreta el importe como descuento.

`feriados_operativos`

Guarda fechas no operativas para el cálculo de días objetivo.

Campos principales:

Campo	Descripción
`year`	Año del feriado.
`fecha`	Fecha exacta.
`nombre`	Nombre o descripción.
`tipo`	Clasificación, por defecto `Manual`.
`activo`	Indica si impacta en cálculos.

Datos que necesitás agregar:

- feriados nacionales, propios o no operativos que deban descontarse;
- una fila por fecha.

Cuando se agrega o elimina un feriado activo, se recalculan las proyecciones del mes afectado.

`matriz_proyecciones`

Guarda la planificación mensual de cada cliente/campaña.

Campos principales:

Campo	Descripción
`cliente`	Cliente proyectado.
`campania`	Campaña proyectada.
`year`	Año de planificación.
`mes`	Mes en formato `YYYY-MM`.
`dotacion_requerida`	Dotación necesaria del mes.

Campo	Descripción
`carga_semanal`	Esquema semanal, por ejemplo `L a V`.
`carga_horaria`	Horas por día o jornada.
`dias_objetivo`	Días operativos del mes.
`horas_requeridas`	Horas esperadas antes de cumplimiento.
`porcentaje_cumplimiento`	Porcentaje aplicado sobre horas requeridas.
`tiene_nocturnidad`	Indica si la proyección se abre en horas diurnas y nocturnas.
`porcentaje_nocturnidad`	Porcentaje nocturno aplicado; en PLP se obtiene desde la distribución mensual.
`tipo_plp`	Clasificación PLP: Personal CX, Personal, Personal Soporte o Personal SMB.
`horas_carga_manual`	Identifica que las horas fueron ingresadas directamente, como ocurre en PLP.
`dias_objetivo_manual`	Reemplazo opcional de los días calculados para campañas que operan solo una parte del mes.
`horas_requeridas_manual`	Reemplazo opcional del total calculado desde dotación, días y jornada.

Clave única:

```
cliente + campania + mes
```

Datos que necesitás agregar:

- una fila por cliente/campaña/mes;
- dotación requerida para usar Control proyecciones en pestaña Dotaciones;
- horas requeridas y porcentaje de cumplimiento para calcular Horas;
- cliente y campaña alineados con `asignaciones_comerciales`.

Esta tabla alimenta Matriz de proyecciones, Control proyecciones y Facturación horas.

En el formulario general, **Días hábiles manuales** y **Horas requeridas manuales** son opcionales. Si quedan vacíos, se usa el cálculo automático. Si se informan, tienen prioridad y permanecen guardados al editar o recalcular la proyección. Esto permite representar altas a mitad de mes, cierres anticipados u otras ventanas operativas parciales.

En PLP el `cliente` siempre se guarda como `Personal`. `tipo_plp` identifica la clasificación y `campania` conserva el nombre operativo importado, por ejemplo `MÓVIL TELEFÓNICO`. En pantalla, todos los servicios de

una misma clasificación y mes se acumulan en dos renglones: la clasificación principal y la misma clasificación con el sufijo `nocturnidad`.

`personal_distribucion_horas`

Campos principales:

Campo	Descripción
`servicio`	Clasificación PLP: Personal CX, Personal, Personal Soporte o Personal SMB.
`year`	Año al que pertenece la distribución.
`mes`	Mes en formato `YYYY-MM`.
`porcentaje_diurno`	Proporción de horas que queda en el renglón principal.
`porcentaje_nocturno`	Valor calculado como `100 - porcentaje_diurno`; no se carga directamente.

Los porcentajes se administran desde **Matriz de proyecciones > PLP**. Cambiar un porcentaje recalcula todas las proyecciones PLP coincidentes del mes. Los datos de años anteriores permanecen intactos.

La tabla **Distribución de horas Personal** muestra los porcentajes diurnos y nocturnos con dos decimales. La precisión histórica almacenada se conserva mientras una celda no sea editada; al modificarla, el paso permitido es de `0,01` puntos porcentuales.

`matriz_proyecciones_jornadas`

Guarda aperturas internas de jornada para una proyección.

Campos principales:

Campo	Descripción
`proyeccion_id`	Proyección principal asociada.
`dotacion_requerida`	Dotación de esa jornada.
`carga_semanal`	Esquema semanal de esa jornada.
`carga_horaria`	Carga horaria de esa jornada.
`dias_objetivo`	Días objetivo de esa jornada.
`horas_requeridas`	Horas requeridas de esa jornada.

Datos que necesitás agregar:

- solo si una campaña tiene más de una jornada o configuración dentro del mismo mes;
- si no hay apertura, alcanza con `matriz_proyecciones`.

`matriz_precios`

Guarda precios mensuales para cruzar contra horas proyectadas.

Campos principales:

Campo	Descripción
`site`	Site/gerencia usado para agrupación.
`cliente`	Cliente.
`campania`	Campaña.
`year`	Año.
`mes`	Mes en formato `YYYY-MM`.
`precio_base`	Precio cargado base.
`alcance_porcentaje`	Porcentaje de alcance aplicado al precio.
`precio_final`	Precio calculado.
`importe_fijo_mensual`	Importe fijo alternativo o adicional del mes.

Clave única:

```
cliente + campania + mes
```

Datos que necesitás agregar:

- una fila por cliente/campaña/mes con precio;
- `alcance_porcentaje` normalmente en `100`;
- `importe_fijo_mensual` en `0` si no aplica;
- precios de todos los meses necesarios para que Facturación horas calcule completo.

Esta tabla alimenta Facturación horas. También recibe impacto de inflación o deflación global desde un mes hacia adelante.

`variables_campanias`

Guarda el porcentaje variable mensual aplicado sobre Facturación horas.

Campos principales:

Campo	Descripción
`site`	Site/gerencia de la campaña.
`cliente`	Cliente.
`campania`	Campaña.
`year`	Año.
`mes`	Mes en formato `YYYY-MM`.
`porcentaje`	Porcentaje variable aplicado.

Clave única:

```
cliente + campania + mes
```

Datos que necesitás agregar:

- una fila por campaña/mes cuando tenga variable;
- se puede guardar `0`;
- la campaña debe existir en Facturación horas para ese año.

Esta tabla alimenta el módulo Variable y puede deshacerse desde Historial.

`tarifaciones_campanias`

Campos principales:

Campo	Descripción
`site`	Site o gerencia de agrupación.
`cliente`	Cuenta comercial.
`campania`	Nombre que debe coincidir con la campaña de Facturación horas.
`concepto`	Etiqueta del ajuste, por ejemplo `Tarifación` o `Penalidad ADH`.

Campo	Descripción
`year`, `mes`	Período del importe.
`monto`	Importe con signo: positivo suma y negativo resta.

La tabla no reemplaza el campo `tarifacion` de la facturación real: `tarifaciones_campanias` pertenece exclusivamente al circuito proyectado y se integra en Facturación horas.

`historial_cambios`

Guarda auditoría de operaciones.

Campos principales:

Campo	Descripción
`usuario_id`, `usuario_nombre`, `usuario_email`	Usuario que ejecutó la acción.
`accion`	Tipo de operación: creación, edición, eliminación, importación, etc.
`entidad`	Módulo afectado.
`entidad_id`	Identificador del registro o movimiento.
`resumen`	Texto corto para mostrar en Historial.
`detalle`	Detalle adicional.
`antes`	Snapshot anterior en JSON.
`despues`	Snapshot posterior en JSON.

Datos que necesitás agregar:

- normalmente ninguno manualmente;
- la aplicación lo completa automáticamente cuando se realizan cambios.

Dependencias entre tablas

Para que los módulos funcionen correctamente:

- `asignaciones_comerciales` debe cargarse antes de proyecciones, precios y variables.
- `matriz_proyecciones` necesita cliente/campaña existentes en el maestro.
- Las proyecciones PLP siempre usan `Personal` como cliente y una de las cuatro clasificaciones admitidas en `tipo_plp`; el nombre operativo queda en `campania`.
- `personal_distribucion_horas` divide las horas PLP en principal y nocturnidad antes del cruce con precios.
- `matriz_precios` debe coincidir en cliente/campaña/mes con `matriz_proyecciones` para que Facturación horas calcule.
- `variables_campanias` se carga sobre campañas que ya aparecen en Facturación horas.
- `tarifaciones_campanias` se cruza por cliente/campaña/mes y suma o resta el monto al consolidado de Facturación horas.
- `feriados_operativos` afecta el cálculo de días objetivo de `matriz_proyecciones`.
- `historial_cambios` no es una tabla de carga manual: audita operaciones del sistema.

Módulos funcionales

Dashboard

Pantalla ejecutiva para seguimiento general.

Permite:

- ver KPIs de facturación;
- consultar total real, total teórico, desvío y cumplimiento;
- analizar evolución mensual;
- revisar alertas;
- filtrar por mes, cliente, gerente, jefe de site, campaña, subcampaña y tipo de negocio;
- abrir lecturas por cliente o por agrupaciones comerciales.

Cargar datos

Módulo para ingresar registros de facturación real.

Incluye:

- carga manual;
- carga masiva desde archivo;
- validación de fechas, importes, horas y datos comerciales;
- detección de registros existentes;
- reemplazo controlado;

- normalización de números con coma o punto;
- soporte para acentos, Ñ y nombres comerciales reales.

La coincidencia para detectar posibles reemplazos se basa en campos comerciales y fecha del registro.

Datos maestros

Administra las asociaciones comerciales que ordenan la información.

Permite mantener:

- cliente;
- gerente;
- jefe de site;
- campaña;
- subcampaña;
- tipo de negocio;
- estado activo/inactivo.

Cuando cambia el jefe de una campaña, permite indicar desde qué mes aplica el cambio para preservar el histórico mensual. Por ejemplo, si `Santander Chat` cambia de `Ismael Rissi` a `Mariela Ditto` desde junio, se edita la asociación, se cambia el jefe y se completa **Aplicar desde mes** con `2026-06`. Los meses anteriores quedan asociados al jefe anterior.

Estas asociaciones alimentan filtros, formularios y agrupaciones de los módulos proyectados.

Control

Vista detallada de registros reales cargados.

Permite:

- buscar registros;
- filtrar por dimensiones comerciales;
- editar datos cargados;
- eliminar registros con control de permisos;
- exportar información;
- revisar importes calculados.

Justificaciones

Módulo para documentar ajustes, desvíos o diferencias.

Permite:

- asociar justificaciones a registros;
- cargar cantidad, precio e importe;
- controlar penalizaciones y ajustes;
- mantener explicación auditable de cambios o diferencias.

Comparativo

Vista de análisis entre objetivo, real y desvíos.

Sirve para:

- comparar cumplimiento;
- revisar diferencias por cliente o campaña;
- detectar desvíos relevantes;
- analizar meses o responsables específicos.

Matriz

Vista matricial de facturación real.

Permite analizar la información cargada en una estructura más compacta, útil para revisión mensual y cruces por dimensiones comerciales.

Proyectados

Los módulos de Proyectados trabajan con planificación futura o estimada. Están pensados para construir un escenario mensual de horas, dotaciones, precios, variables y facturación esperada.

Matriz de proyecciones

Carga la planificación mensual por cliente y campaña.

Campos principales:

- año;
- mes;
- cliente;
- campaña;
- dotación requerida;
- carga semanal;

- carga horaria;
- días objetivo;
- horas requeridas;
- porcentaje de cumplimiento;
- horas proyectadas.

Regla principal:

```
horas proyectadas = horas requeridas * porcentaje de cumplimiento / 100
```

El módulo también considera calendario operativo y feriados para recalcular días objetivo cuando corresponde.

Funciones disponibles:

- carga manual por campaña;
- importación desde plantilla;
- descarga de plantilla;
- eliminación individual;
- eliminación masiva del año;
- deshacer último cambio;
- registro en historial.

Subpestaña PLP

Es el circuito exclusivo para campañas cuyo cliente es `Personal`. Las clasificaciones válidas son `Personal CX`, `Personal`, `Personal Soporte` y `Personal SMB`.

Cada registro conserva el nombre operativo de la campaña, el mes y las horas cargadas, pero el listado acumula los nombres operativos de una misma clasificación. La apertura se muestra en dos renglones:

- clasificación principal: horas correspondientes al porcentaje diurno;
- `clasificación nocturnidad`: horas restantes hasta completar el 100%.

La jornada comienza predefinida en `L a V` y `6` horas, pero ambos campos son editables. Los días objetivo salen de los días laborales del mes, descontando feriados activos. La dotación se recalcula así:

```
dotación requerida = horas requeridas / días laborales / horas de jornada
```

El porcentaje de cumplimiento es editable y afecta las horas proyectadas finales. El resumen mensual PLP acumula las horas base importadas al 100% para poder cotejarlas contra el archivo original.

El **Resumen mensual PLP** aparece antes de la edición para cotejar Personal CX, Personal, Personal Soporte, Total Personal, porcentaje de Soporte, Personal SMB y Total contra el archivo de control. Las horas y los totales se muestran siempre con dos decimales y se calculan sin redondear previamente cada clasificación. La sección **Edición rápida de horas PLP** presenta luego una grilla tipo planilla con una fila por campaña. Su

filtro **Mes** permite trabajar con un período puntual o seleccionar **Todos los meses**, y limita tanto el resumen como la grilla. Permite modificar clasificación, mes, campaña, horas, carga semanal, jornada y cumplimiento en varias filas antes de presionar **Guardar cambios** una sola vez. Al finalizar correctamente, la pantalla muestra una confirmación explícita con la cantidad de filas guardadas y eliminadas. **Agregar fila** incorpora campañas nuevas sin abandonar la grilla. Si la combinación clasificación + campaña + mes ya existe, sus valores se reemplazan; no se crea una fila duplicada. **Eliminar** marca una campaña existente para quitarla y ofrece **Restaurar** antes de guardar; las filas nuevas se retiran inmediatamente. La operación es transaccional: si alguna fila es inválida o la misma clave aparece repetida dentro del lote enviado, ninguna modificación ni eliminación se confirma.

Para una conciliación anual controlada puede ejecutarse `python scripts/sincronizar_porcentajes_personal.py RUTA_EXCEL --year AAAA`. El proceso lee las hojas `Desglose HORAS PERSONAL`, `Horas`, `Precios`, `Variable` y `Tarifacion`; conserva la precisión interna del archivo, recalcula distribuciones y porcentajes y registra el cambio en Historial. No genera ajustes residuales.

La importación anual acepta `.xlsx` o `.csv` con estas columnas:

```
Clasificación PLP | MES | CAMPAÑA | HORAS | Carga semanal | Horas jornada | % cumplimiento
```

Si se vuelve a importar la misma combinación `clasificación + campaña + mes`, se actualiza el registro existente en lugar de duplicarlo.

Calendario operativo

Administra feriados o días no operativos.

Permite:

- cargar feriados por fecha;
- indicar descripción;
- activar o eliminar feriados;
- recalcular proyecciones afectadas;
- impactar en días objetivo y horas proyectadas.

Cuando se agrega o elimina un feriado, el sistema recalcula las proyecciones del mes afectado.

Matriz de precios

Carga precios mensuales por campaña.

Permite:

- cargar precio por cliente/campaña/mes;
- cargar importe fijo mensual;
- importar desde plantilla;
- descargar plantilla;

- eliminar precios;
- aplicar inflación global desde un mes hacia adelante;
- usar índices positivos, cero o negativos.

Regla de inflación:

```
nuevo precio = precio actual + (precio actual * índice / 100)
```

El índice se aplica a todos los precios desde el mes elegido hacia adelante. Si el índice es negativo, funciona como deflación. Si es `0`, mantiene los precios sin cambios pero registra la operación cuando corresponde.

Los cálculos monetarios usan redondeo consistente para evitar diferencias entre campañas con el mismo precio base.

Facturación horas

Antes llamada Resumen. Muestra el cruce entre proyecciones y precios.

Calcula:

```
facturación horas = horas proyectadas * precio mensual
```

También contempla importes fijos cuando están cargados.

La vista final consolida en una sola línea por campaña:

```
total campaña = facturación de horas + estimación variable + tarifaciones
```

En PLP, las horas principales y nocturnas se calculan con su precio correspondiente y luego se consolidan bajo la clasificación base. La asociación de variables y tarifaciones ignora diferencias de mayúsculas, minúsculas y acentos. `Multicuentas` y `Gerencia Multicampaña` se presentan bajo la misma agrupación comercial.

La identificación es editable directamente desde el lápiz de la columna **Site**. Allí se pueden regularizar el site, el cliente y la campaña. El cliente/campaña ingresado funciona como nombre canónico: si varias identificaciones de origen se asignan al mismo nombre, sus Horas, Variable, Tarifación y Next Gen se consolidan y suman en una única línea para cada mes. La regularización se guarda en `sites_proyecciones`, no modifica los nombres históricos de las tablas de origen, queda registrada en Historial y puede deshacerse.

Las proyecciones comunes se calculan por cada apertura de campaña, respetando su precio y porcentaje variable, y después se consolidan bajo el cliente. Si una apertura no tiene proyección propia para un mes, se utiliza la proyección base del cliente como respaldo. PLP conserva como excepción sus clasificaciones Personal, Personal CX, Personal Soporte y Personal SMB.

Los importes de **Suma fija** se guardan como `importe_fijo_mensual` dentro de `matriz_precios`. Facturación horas los incorpora como concepto `Fijo mensual`, separado de Horas, Variable, Tarifación y Next Gen. Para 2026 incluye los importes por dotación de Assurant, BBVA Seguros, OMINT y Qualia Cencosud, además de los importes fijos propios que ya existan, como Bice.

Permite:

- filtrar por site, cliente, campaña y concepto;
- ver totales anuales;
- ver evolución mensual;
- revisar detalle por campaña;
- identificar campañas sin precio o sin cruce;
- usar la información como base para el cálculo de Variable.

Variable

El módulo tiene tres subpestañas relacionadas, pero cada una conserva su propio dato:

- **% Variable**: porcentaje mensual guardado en `variables_campanias`.
- **Estimación variable**: resultado monetario calculado; no es otra carga independiente.
- **Tarifación**: monto mensual positivo o negativo guardado en `tarifaciones_campanias`.
- **Next Gen**: cantidades mensuales en USD por producto, valorizadas con el dólar de cada mes.

La subpestaña **% Variable** permite cargar un porcentaje mensual por campaña sobre el resultado de Facturación horas.

La pantalla muestra solo campañas que existen en Facturación horas, para evitar cargar variables sobre campañas sin base de cálculo.

Campos:

- campaña;
- mes;
- porcentaje variable.

Regla:

```
resultado variable = facturación horas del mes * porcentaje variable / 100
```

Características:

- el porcentaje puede ser `0`;
- el input selecciona el valor al hacer foco para facilitar la carga;
- los resultados se muestran por mes;
- los números se muestran centrados;
- se puede deshacer desde Historial;

- soporta acentos y Ñ en cliente, campaña y site.

La plantilla anual de variables contiene `Cuenta`, `Site`, `Cliente` y una columna por mes. Acepta porcentajes como `5`, `5%` o el valor decimal de Excel `0,05`. Al reimportar, se actualiza la combinación de campaña y mes existente. Al seleccionar un año nuevo sin valores propios, se toman como propuesta los últimos porcentajes disponibles del año anterior; los cambios posteriores no alteran el histórico.

Estimación variable

No requiere una tabla adicional. Se calcula sobre la facturación de horas de la misma campaña y mes:

```
estimación variable = facturación horas * porcentaje / 100
```

Para PLP, las campañas principales y nocturnas pueden tener porcentajes propios. El resultado se consolida finalmente en el renglón de su clasificación base.

Tarifación

Permite ingresar o importar un monto adicional por campaña, concepto y mes:

```
facturación consolidada = horas + estimación variable + monto de tarifación
```

Los montos negativos restan y los positivos suman. La plantilla anual incluye `Cuenta`, `Site`, `Cliente`, `Concepto` y los doce meses. Reimportar una misma combinación actualiza el valor existente.

Next Gen

Permite editar el valor mensual del dólar y cargar uno o varios productos por campaña. Cada producto conserva su cantidad mensual en USD y muestra el resultado convertido a pesos:

```
Next Gen en pesos = cantidad USD del producto * valor del dólar del mes
```

Los productos de una misma campaña se acumulan y el resultado se incorpora como concepto `Next Gen` en Facturación horas. La plantilla anual replica el cuadro operativo: una columna `CLIENTE` para identificar el producto y doce columnas mensuales en USD. El valor del dólar se administra por mes desde la parte superior de la pestaña. Al reimportar, el sistema conserva la campaña asociada al producto y actualiza cada mes existente.

Control proyecciones

Módulo separado de Variable para revisar lo cargado en Matriz de proyecciones.

Tiene dos pestañas:

- **Horas**: muestra horas proyectadas por campaña y mes.
- **Dotaciones**: muestra dotación requerida por campaña y mes.

La información se trae directo desde las proyecciones cargadas. No calcula facturación ni variables: es una vista de control operativo para validar que la planificación mensual esté bien cargada.

Permite:

- filtrar por site;
- filtrar por cliente;
- filtrar por campaña;
- buscar por texto;
- ver total anual;
- ver totales por mes;
- alternar entre horas y dotaciones sin salir del módulo.

Historial y deshacer

El sistema registra cambios relevantes en `HistorialCambio`.

Se registran operaciones como:

- creación;
- edición;
- eliminación;
- importación;
- eliminación masiva;
- inflación aplicada;
- cambios de variable;
- cambios de proyecciones;
- cambios de precios.
- cambios e importaciones de porcentajes variables;
- cambios e importaciones de tarifación;
- importaciones PLP y cambios de distribución diurna/nocturna.

Desde Historial se puede deshacer, cuando aplica, cambios de:

- Matriz de proyecciones;
- Matriz de precios;
- Variable.
- Tarifación.

El deshacer restaura el estado anterior guardado en el snapshot del historial. Si el registro no existía, se recrea; si debía eliminarse, se elimina; si debía modificarse, se restaura el valor previo.

Importaciones y plantillas

El sistema ofrece plantillas para evitar errores de estructura.

Plantillas disponibles:

- carga de facturación;
- matriz de proyecciones;
- matriz de precios.
- proyecciones anuales PLP;
- porcentajes variables anuales;
- tarifaciones anuales.

Destino de cada importación:

Plantilla	Tabla principal	Pantalla de destino	Resultado posterior
Facturación real	`facturacion_anio`	Cargar datos / Control	Dashboard, Comparativo, Matriz y exportaciones.
Matriz de proyecciones	`matriz_proyecciones` y, si aplica, `matriz_proyecciones_jornadas`	Proyectados > Matriz de proyecciones	Control proyecciones y Facturación horas.
Proyecciones PLP	`matriz_proyecciones` con `cliente=Personal` y `tipo_plp`	Proyectados > Matriz de proyecciones > PLP	Resumen PLP, aperturas diurnas/nocturnas y Facturación horas.
Matriz de precios	`matriz_precios`	Proyectados > Matriz de precios	Precio de las horas y de las aperturas nocturnas.
Variables	`variables_campanias`	Variable > % Variable	Variable > Estimación variable y Facturación horas.
Tarifación	`tarifaciones_campanias`	Variable > Tarifación	Suma o resta en Facturación horas.
Next Gen	`next_gen_dolar` y `next_gen_productos`	Variable > Next Gen	Convierte USD a pesos y suma el resultado en Facturación horas.

Formatos soportados según el módulo:

- `.xlsx` ;

- `.csv`;
- HTML tabular copiado desde Excel, cuando el parser lo permite.

Las importaciones validan encabezados, meses y campos obligatorios antes de guardar.

Exportaciones

La aplicación permite exportar información a Excel desde módulos de control y análisis.

Las exportaciones respetan filtros activos cuando el endpoint lo contempla. La descarga completa de **Directorio** genera un libro de Excel con una hoja por solapa o vista disponible. Los valores se exportan como datos editables, con títulos y estructura legible. El reloj también puede descargarse como imagen desde su propia pantalla.

Los números visibles siguen la convención argentina: punto para miles y coma para decimales. Los archivos tabulares usan valores numéricos o representaciones compatibles con Excel según el formato de descarga.

Proyectado vs Facturado y normalización de nombres

La vista **Directorio > Proyectado vs Facturado** compara por año y mes tres métricas: teórico, proyectado y facturado. Permite filtrar por site o gerencia, cliente, servicio y texto libre. El gráfico incluye leyendas de métricas y servicios, etiquetas resumidas dentro del gráfico y detalle completo al pasar el cursor.

La comparación aplica las correspondencias configuradas en **Datos maestros > Normalizar nombres proyectados**. El usuario selecciona primero el nombre proveniente de Proyectados y luego vincula una o varias campañas canónicas de Facturación. La relación se guarda en `normalizacion_nombres_proyectados`, se refleja inmediatamente en la vista y también en la exportación de Directorio. Los importes y nombres de origen permanecen sin modificación.

Cada `campania_id` de Facturación puede tener una sola correspondencia activa. Cambiar una asociación mueve esa campaña al nombre proyectado elegido; quitarla vuelve a dejarla pendiente de normalización.

Cálculos principales

Facturación real

El modelo de facturación calcula, entre otros:

- valor hora alcanzado;
- valor hora objetivo;
- objetivo de facturación horas;
- objetivo de facturación bono;
- facturado horas;
- facturado bono;
- variable productivo;
- penalizaciones;
- total real;
- total teórico;
- desvío;
- porcentaje de cumplimiento.

Penalizaciones

Las penalizaciones impactan como descuento. Aunque se carguen como valor positivo, el cálculo las resta del total correspondiente.

Redondeo monetario

Los importes monetarios proyectados usan redondeo centralizado para evitar diferencias pequeñas entre campañas con el mismo precio base.

Proyección PLP

```
horas diurnas = horas base * porcentaje diurno / 100
horas nocturnas = horas base * (100 - porcentaje diurno) / 100
horas proyectadas finales = horas base * porcentaje de cumplimiento / 100
dotación = horas base / días laborales / horas de jornada
```

Las aperturas diurna y nocturna se valorizan con el precio mensual que coincida con cada nombre. El resumen operativo PLP muestra las horas base al 100%; el cumplimiento se aplica en el circuito de

proyección y facturación.

Consolidado proyectado

```
importe horas = horas proyectadas * precio final
estimación variable = importe horas * porcentaje variable / 100
total campaña = importe horas + estimación variable + Next Gen + tarifaciones
```

Una tarifación negativa funciona como descuento. No se debe confundir con el porcentaje variable ni con el campo de tarifación de la facturación real.

APIs principales

Rutas HTML:

```
/login
/
/cargar
/carga-datos-personal
/catalogos
/control
/justificaciones
/comparativo
/matriz
/matriz-proyecciones
/matriz-precios
/suma-fija
/resumen
/variable
/control-proyecciones
/calendario-operativo
/historial
/usuarios
/proyectado-vs-facturado
```

APIs destacadas:

```
/api/auth/login
/api/auth/logout
/api/auth/me
/api/datos
/api/cargar
/api/resumen
```

```
/api/kpis
/api/grafico
/api/filtros
/api/comparativo
/api/matriz
/api/matriz-proyecciones
/api/proforma-personal
/api/proforma-personal/template
/api/proforma-personal/importar
/api/seguimiento-personal
/api/proyecciones-plp/template
/api/proyecciones-plp/importar
/api/personal-distribucion
/api/matriz-precios
/api/matriz-precios/inflacion
/api/suma-fija
/api/resumen-proyeccion
/api/variables
/api/variables/template
/api/variables/importar
/api/tarifaciones
/api/tarifaciones/template
/api/tarifaciones/importar
/api/next-gen
/api/next-gen/template
/api/next-gen/importar
/api/sites-proyeccion
/api/control-proyecciones
/api/calendario-operativo
/api/historial
/api/historial/<id>/deshacer
/api/exportar_excel
/api/template_carga
/api/importar_datos
/api/asignaciones
/api/clientes
/api/gerentes
/api/meses
/api/proyectado-vs-facturado
/api/catalogos/normalizaciones-proyectados
/api/catalogos/normalizaciones-proyectados/<id>
/api/directorio/exportar
```

Seguridad

Medidas incluidas:

- Flask escucha en `127.0.0.1` por defecto;

- sesiones con cookie `HttpOnly`;
- `SameSite=Lax`;
- CSRF obligatorio en métodos de escritura;
- validación de `Origin` / `Referer`;
- CORS limitado a orígenes configurados;
- límite de tamaño de archivo;
- rate limit simple para intentos de login;
- headers de seguridad;
- permisos por rol;
- confirmaciones para acciones sensibles.

> Para producción:

```
SECRET_KEY=generar-una-clave-segura
SESSION_COOKIE_SECURE=1
FLASK_DEBUG=0
FLASK_RELOAD=0
TEMPLATES_AUTO_RELOAD=0
```

Servir siempre detrás de HTTPS y con una base de datos administrada.

Flujo recomendado de uso

Carga posterior de un mes anterior

La fecha en que se sube un archivo no define el período contable. Si en septiembre se carga información cuyo campo `Mes` o `Fecha` corresponde a agosto, se persiste como `YYYY-08` y se incorpora a agosto en Dashboard, Control, Histórico, Directorio, comparativos e indicadores acumulados.

Las claves mensuales mantienen `year` alineado con el prefijo de `mes`. Las reimportaciones actualizan la misma clave; cuando una campaña contiene varias líneas aditivas con idénticas dimensiones, el conjunto anterior se reemplaza una sola vez por el conjunto completo recibido. Esto evita duplicar o colapsar una parte del total. Las variantes de mayúsculas/minúsculas de un cliente se consolidan al responder las APIs de Histórico sin alterar los importes fuente.

Generación de un nuevo año proyectado

La acción **Nuevo** de Matriz de proyecciones toma el año seleccionado en pantalla como base y, dentro de ese año, el último mes con datos disponible hasta el mes calendario actual. No toma meses futuros aunque

ya tengan una previsión cargada: en agosto usa agosto y, cuando septiembre esté cargado, usa septiembre. Esa fotografía mensual se replica en los doce meses del año siguiente para proyecciones, jornadas, precios, suma fija, porcentajes variables y distribución horaria de Personal.

Los días objetivo y las horas derivadas se recalculan con el calendario de cada mes destino. Los feriados no se heredan y deben cargarse expresamente para el nuevo año. La operación es atómica, queda en el historial y rechaza la solicitud si el año destino ya tiene proyecciones, evitando sobrescrituras parciales.

1. Cargar o revisar Datos maestros.
2. Cargar facturación real en Cargar datos.
3. Controlar registros desde Control.
4. Revisar Dashboard, Comparativo y Matriz.
5. Cargar Matriz de proyecciones.
6. Configurar Calendario operativo.
7. Para Personal, seleccionar el año, revisar la distribución diurna/nocturna e importar las proyecciones PLP.
8. Cargar Matriz de precios, incluyendo precios nocturnos cuando correspondan.
9. Revisar Facturación horas.
10. Descargar o importar la plantilla de `% Variable` y revisar Estimación variable.
11. Cargar o importar Tarifación con el signo correspondiente.
12. Volver a Facturación horas y validar el consolidado `Horas + Variable + Tarifación`.
13. Revisar Control proyecciones para validar horas y dotaciones.
14. Usar Historial para auditar o deshacer cambios.

Problemas frecuentes

`npm` no se reconoce

Instalar Node.js o ejecutar:

```
.\dev.ps1
```

La app usa otro puerto

Configurar `PORT` antes de iniciar Flask:

```
$env:PORT="8010"
python run.py
```

No veo cambios en templates

En desarrollo, activar recarga:

```
$env:FLASK_RELOAD="1"
$env:TEMPLATES_AUTO_RELOAD="1"
python run.py
```

O reiniciar el servidor.

Error de permisos con SQLite

Verificar que el proceso que ejecuta Flask tenga permiso de escritura sobre:

```
instance/facturacion.db
```

No puedo guardar desde el navegador

Revisar:

- sesión iniciada;
- permisos del rol;
- token CSRF presente;
- origen permitido en `CORS_ORIGINS`;
- que se esté usando `127.0.0.1` o `localhost` según configuración.

Notas de desarrollo

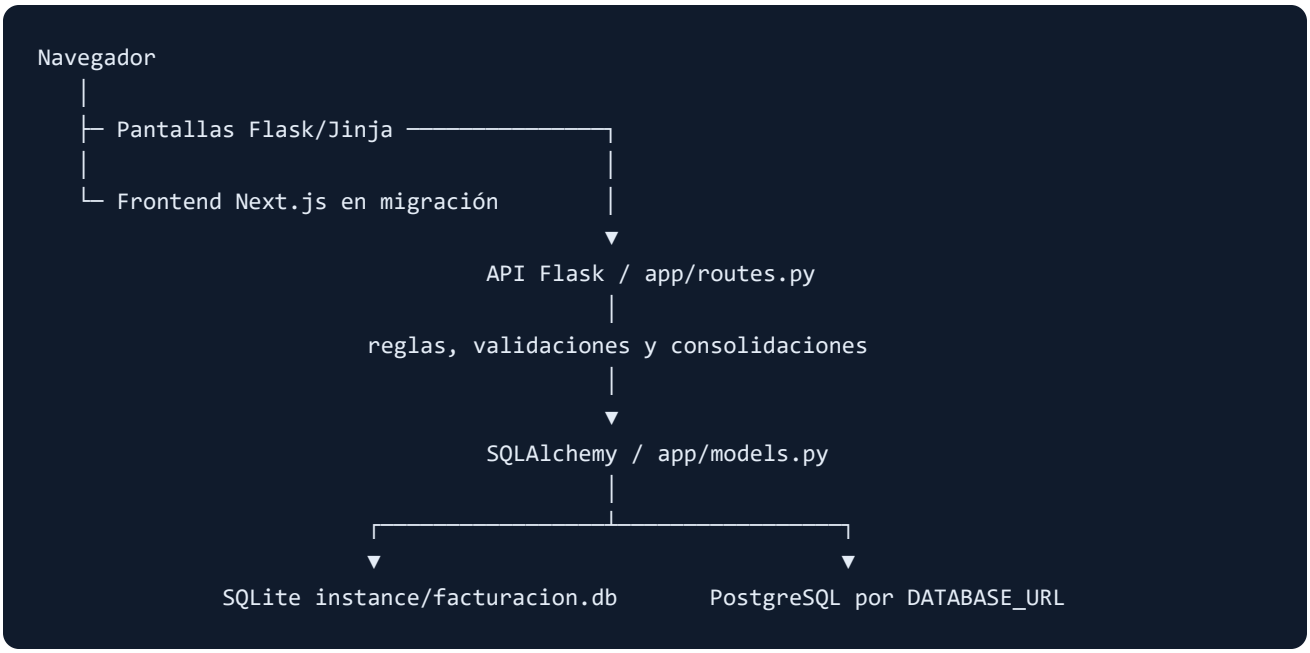
- El backend está concentrado en `app/routes.py` y `app/models.py`.
- La base local se crea y ajusta automáticamente al iniciar la app.
- Las pantallas Flask usan templates Jinja en `app/templates`.
- El frontend Next.js vive en `next-app`.
- Evitar editar manualmente la base salvo que sea necesario.
- Mantener textos visibles en español con acentos y Ñ.
- No dejar cambios de `__pycache__` ni logs como parte de commits.

Guía técnica integral para mantenimiento y transferencia

Esta sección describe el estado técnico actual de la aplicación. Su objetivo es que una persona que no participó del desarrollo pueda identificar rápidamente:

- dónde se almacena cada dato;
- cuál es la fuente maestra;
- qué pantallas y APIs lo modifican;
- cómo se construyen los resultados;
- qué relaciones dependen de coincidencias de nombres;
- qué cambios son auditables y reversibles;
- cómo diagnosticar una diferencia contra el archivo de planificación.

Arquitectura de ejecución



El backend Flask es la implementación operativa principal. Las rutas HTML y API están registradas sobre `main_bp` en `app/routes.py`. SQLAlchemy administra los modelos y la conexión. `create_app()` en `app/__init__.py` configura seguridad, sesión, CORS, CSRF, base de datos y ajustes simples de esquema.

`run.py` inicia Flask usando:

```
FLASK_HOST    dirección de escucha; 127.0.0.1 por defecto
PORT          puerto; 8009 por defecto
```

```
FLASK_DEBUG   activa debug cuando vale 1
FLASK_RELOAD  activa el reloader cuando vale 1
```

El script `scripts/dev.mjs`, invocado con `npm run dev`, administra Flask y Next.js en desarrollo.

Propiedad y flujo de los datos

Dato	Fuente maestra en la app	Pantalla principal	Resultado consumidor
Usuarios y roles	`usuarios`	Usuarios	Autenticación y permisos
Catálogo comercial	`asignaciones_comerciales`	Datos maestros	Formularios, sites y filtros
Facturación real	`facturacion_anio`	Cargar datos / Control	Dashboard, Comparativo y Matriz
Justificaciones reales	`justificaciones_ajustes`	Justificaciones	Total real y trazabilidad
Proyección de horas	`matriz_proyecciones`	Proyecciones	Control proyecciones y Facturación horas
Jornadas múltiples	`matriz_proyecciones_jornadas`	Proyecciones	Cálculo de horas requeridas
Distribución PLP	`personal_distribucion_horas`	Proyecciones / PLP	Apertura diurna y nocturna
Precio mensual	`matriz_precios.precio_final`	Precios	Facturación horas
Suma fija	`matriz_precios.importe_fijo_mensual`	Variable / Suma fija	Facturación horas
Porcentaje variable	`variables_campanias`	Variable / % Variable	Estimación variable y Facturación horas
Tarifación y adicionales	`tarifaciones_campanias`	Variable / Tarifación	Facturación horas
Dólar Next Gen	`next_gen_dolar`	Variable / Next Gen	Valorización Next Gen
Cantidades Next Gen	`next_gen_productos`	Variable / Next Gen	Facturación horas
Alias y consolidación	`sites_proyecciones`	Lápiz de Facturación horas	Site, cliente y campaña canónicos
Feriados	`feriados_operativos`	Calendario operativo	Días objetivo

Dato	Fuente maestra en la app	Pantalla principal	Resultado consumidor
Auditoría	`historial_cambios`	Historial	Consulta y deshacer

Una salida calculada, como Facturación horas, no debe importarse nuevamente como si fuera una fuente maestra. Deben cargarse sus componentes: horas, precios, variables, sumas fijas, tarifaciones y Next Gen.

Diccionario completo de tablas

`usuarios`

Modelo: `Usuario`.

Campo	Tipo lógico	Uso
`id`	entero	Clave primaria
`nombre`	texto	Nombre mostrado
`email`	texto único	Identificador de login
`password_hash`	texto	Hash seguro de contraseña
`rol`	texto	Rol y permisos
`activo`	booleano	Habilita el acceso
`creado_en`, `actualizado_en`	fecha/hora	Auditoría técnica

No se almacena la contraseña original. La baja lógica se realiza con `activo`.

`historial_cambios`

Modelo: `HistorialCambio`.

Campo	Uso
`usuario_id`, `usuario_nombre`, `usuario_email`	Autor de la operación
`accion`	Creación, edición, eliminación, importación o deshacer
`entidad`	Familia técnica afectada
`entidad_id`	Identificador cuando existe uno único

Campo	Uso
`resumen`	Texto corto mostrado en Historial
`detalle`	Explicación funcional
`antes`	Snapshot JSON anterior
`despues`	Snapshot JSON posterior
`creado_en`	Momento de la operación

Las entidades reversibles actuales incluyen proyecciones, precios, variables, tarifaciones, Next Gen y regularizaciones de Facturación horas. Un deshacer aplica el snapshot anterior y crea otro movimiento de historial; no elimina la auditoría.

`facturacion_anio`

Modelo: `FacturacionAnio`.

Contiene el escenario real. Sus dimensiones son fecha, mes, cliente, gerente, jefe de site, campaña, subcampaña, tipo de negocio y jornada. Sus medidas incluyen horas objetivo, horas facturadas, horas penalizadas, valor hora, tarifación, importe fijo, variables, bonos, penalizaciones, Next Gen y otros.

La tabla se llamó anteriormente `facturacion_2026`. Durante el primer arranque posterior al cambio, la aplicación ejecuta un `ALTER TABLE ... RENAME TO facturacion_anio`, conservando IDs, registros y la referencia `justificaciones_ajustes.facturacion_id`. Si las dos tablas existieran al mismo tiempo, el inicio se detiene para exigir una conciliación manual y evitar pérdida o duplicación de datos.

Las propiedades calculadas del modelo producen:

```
facturado horas
facturado bono
variable productivo
penalizaciones
total real
total teórico
desvío
porcentaje de cumplimiento
```

Una penalización positiva cargada por el usuario se convierte en descuento durante el cálculo.

`justificaciones_ajustes`

Modelo: `JustificacionAjuste`.

Cada registro referencia `facturacion_anio.id`. Guarda tipo, cantidad, precio, importe y descripción. Su finalidad es explicar y respaldar ajustes del escenario real. Las penalizaciones se normalizan con signo

negativo al calcular.

`matriz_proyecciones`

Modelo: `ProyeccionMatriz`.

Clave funcional:

```
cliente + campaña + mes
```

Campo	Uso
`cliente`, `campania`	Identidad de la apertura
`year`, `mes`	Período
`dotacion_requerida`	Personas requeridas
`carga_semanal`	Días operativos: L a V, L a V + S, etc.
`carga_horaria`	Horas diarias
`dias_objetivo`	Días laborales del mes
`horas_requeridas`	Base de horas
`porcentaje_cumplimiento`	Factor aplicado a las horas
`tiene_nocturnidad`	Indica apertura nocturna
`porcentaje_nocturnidad`	Parte nocturna
`tipo_plp`	Personal, Personal CX, Personal Soporte o Personal SMB
`horas_carga_manual`	Indica que las horas se ingresaron manualmente
`dias_objetivo_manual`	Reemplazo manual de días
`horas_requeridas_manual`	Reemplazo manual de horas

Propiedad principal:

```
horas proyectadas = horas requeridas × porcentaje cumplimiento / 100
```

Cuando una campaña empieza o termina a mitad de mes se deben usar días u horas manuales. Esto evita que el calendario completo fuerce un valor incorrecto.

`matriz_proyecciones_jornadas`

Modelo: `ProyeccionMatrizJornada`.

Permite que una proyección tenga varias jornadas. Cada fila almacena dotación, carga semanal, carga horaria, días objetivo y horas requeridas. La relación usa `proyeccion_id` y eliminación en cascada.

`personal_distribucion_horas`

Modelo: `PersonalDistribucionHoras`.

Clave:

```
servicio + mes
```

Guarda el porcentaje diurno para cada clasificación PLP. El porcentaje nocturno es calculado:

```
porcentaje nocturno = 100 - porcentaje diurno
```

Los porcentajes pertenecen a un año. Crear 2027 puede copiar inicialmente 2026, pero los cambios de 2027 no deben modificar 2026.

`matriz_precios`

Modelo: `ProyeccionPrecio`.

Clave:

```
cliente + campaña + mes
```

Campo	Uso
`site`	Gerencia o site
`cliente`, `campania`	Identidad del precio
`year`, `mes`	Período
`precio_base`	Precio ingresado
`alcance_porcentaje`	Porcentaje aplicado
`precio_final`	Precio monetario calculado

Campo	Uso
`importe_fijo_mensual`	Suma fija asociada a dotación

```
precio final = precio base × alcance / 100
```

Una inflación global modifica precios desde el mes seleccionado hacia adelante. La suma fija no es un precio por hora: se suma una sola vez en el mes.

`variables_campanias`

Modelo: `VariableCampania`.

Guarda un porcentaje por cliente, campaña y mes. Puede ser positivo, negativo o cero. Para un año nuevo se puede partir del último año disponible y modificar manualmente las excepciones.

```
importe variable = facturación de horas × porcentaje / 100
```

El porcentaje se aplica sobre el componente Horas de su apertura antes de la consolidación canónica. Esto permite que dos aperturas del mismo cliente tengan porcentajes diferentes.

`tarifaciones_campanias`

Modelo: `TarifacionCampania`.

Clave:

```
cliente + campaña + concepto + mes
```

El monto conserva su signo:

- positivo: suma;
- negativo: resta;
- cero: no altera el resultado.

Solo deben almacenarse conceptos e importes presentes en la fuente operativa. No se deben crear residuales técnicos para forzar la conciliación: cualquier diferencia debe corregirse en horas, precios, porcentajes variables o tarifaciones de origen.

Si cambian datos maestros tanto en la app como en el Excel, se debe volver a ejecutar una comparación y recalcular la conciliación. No debe editarse el ajuste para ocultar un error de horas, precio o agrupación.

`next_gen_dolar`

Modelo: `NextGenDolar`.

Una fila por mes. Guarda el valor del dólar usado para convertir productos Next Gen a pesos. El año está incluido explícitamente y el mes usa `YYYY-MM`.

`next_gen_productos`

Modelo: `NextGenProducto`.

Guarda site, cliente, campaña, producto, año, mes y cantidad USD.

```
Next Gen en pesos = cantidad USD × dólar del mes
```

Un producto debe consolidarse en la cuenta original, no aparecer como una cuenta independiente llamada Next Gen.

`sites_proyecciones`

Modelo: `SiteProyeccion`.

Es una tabla de normalización manual. Conserva:

Campo	Uso
`cliente`, `campania`	Identidad original
`site`	Site regularizado
`cliente_destino`	Cliente canónico
`campania_destino`	Campaña/cuenta canónica

Facturación horas consulta esta tabla antes de consolidar. Si dos orígenes apuntan al mismo cliente y campaña destino, sus componentes se suman en una sola línea. Los datos históricos originales no se renombran.

Ejemplos de consolidación:

```
Santander Getnet Supervisor exclusivo → Santander Getnet
aperturas Naturgy BAN                 → Naturgy
aperturas Naturgy NOA                 → GASNOR
aperturas Energía San Juan            → ENERGIA SAN JUAN
```

`feriados_operativos`

Modelo: `FeriadoOperativo`.

Guarda año, fecha, nombre, tipo y estado activo. Sólo los feriados activos afectan los días objetivo. Agregar o eliminar uno recalcula las proyecciones del mes alcanzado.

`asignaciones_comerciales`

Modelo: `AsignacionComercial`.

Es el catálogo transversal de cliente, gerente, jefe de site, campaña, subcampaña, tipo de negocio y estado. Debe mantenerse consistente con las cargas. La edición con “Aplicar desde mes” protege el histórico anterior.

Pipeline exacto de Facturación horas

El endpoint `/api/resumen-proyeccion` ejecuta conceptualmente este flujo:

```
Proyección de cada apertura
        │
        ├─ horas proyectadas × precio final ──────────────► Horas
        │
        ├─ Horas × porcentaje variable ──────────────────► Variable
        │
        ├─ importe_fijo_mensual ─────────────────────────► Fijo mensual
        │
        ├─ tarifaciones_campanias ───────────────────────► Tarifación/adicional
        │
        └─ cantidad USD × dólar mensual ─────────────────► Next Gen
                                                              │
                                                              ▼
                                        regularización de site/cuenta/campaña
                                                              │
                                                              ▼
                                           consolidado por cuenta y por mes
```

Fórmula del total:

```
total mensual de cuenta
  = Horas
  + Variable
  + Fijo mensual
  + Tarifación y otros conceptos
  + Next Gen
```

La comparación de nombres ignora mayúsculas, minúsculas y acentos cuando la regla lo permite. La nocturnidad se reconoce como una apertura de la clasificación base.

Para una apertura que no tenga proyección propia en un mes, el cálculo puede usar la proyección base del cliente cuando la relación apertura-cliente fue aprendida de otros meses. Esta regla resuelve, por ejemplo,

aperturas que empiezan después de enero sin perder el cálculo del primer mes.

Tratamiento especial PLP

PLP identifica exclusivamente al cliente Personal y mantiene cuatro clasificaciones:

```
Personal
Personal CX
Personal Soporte
Personal SMB
```

Las campañas operativas se cargan debajo de una clasificación. El resumen PLP suma todas las campañas de una misma clasificación. Si existe nocturnidad:

```
horas diurnas   = horas totales × porcentaje diurno
horas nocturnas = horas totales × porcentaje nocturno
```

El cumplimiento afecta las horas finales de Facturación horas, pero el cuadro de control PLP puede mostrar la carga original al 100% cuando su finalidad es reconciliar el archivo operativo.

La dotación informativa se calcula como:

```
dotación = horas / días laborales / horas de jornada
```

Los valores predeterminados son Lunes a Viernes y seis horas por jornada, pero son editables.

Suma fija

La subpestaña está en:

```
Proyectados → Variable → Suma fija
```

Ruta:

```
/suma-fija
```

La pantalla no crea una tabla duplicada. Edita directamente `matriz_precios.importe_fijo_mensual`. Cada celda corresponde a un registro de precio existente para la campaña y el mes.

Comportamiento:

- el cambio se guarda al salir del campo;

- sólo se permiten valores no negativos;
- Facturación horas lo incorpora como `Fijo mensual`;
- el movimiento queda en Historial bajo `matriz_precios`;
- deshacer restaura el valor anterior.

Para 2026 se cargaron desde la hoja `Suma fija` los importes por dotación de Assurant, BBVA Seguros, OMINT y Qualia Cencosud. Bice conserva su propio importe fijo ya existente.

Correspondencia con el Excel de planificación

El libro de referencia usado durante la alineación contiene hojas maestras y hojas calculadas.

Hoja Excel	Naturaleza	Destino en la app
`Matriz`	Maestra	`matriz_proyecciones`
`Precios`	Maestra	`matriz_precios`
`Suma fija`	Maestra	`matriz_precios.importe_fijo_mensual`
`Estimacion Variable`	Maestra	`variables_campanias`
`Tarifacion`	Maestra	`tarifaciones_campanias`
`Valores Next Gen`	Maestra	`next_gen_dolar` y `next_gen_productos`
`Facturacion Horas`	Calculada	Validación de Horas
`Variable`	Calculada	Validación del importe variable
`Next Gen`	Calculada	Validación de conversión a pesos
`RESUMEN`	Calculada	Validación final por cuenta/mes

No se deben importar directamente `Facturacion Horas`, `Variable`, `Next Gen` o `RESUMEN` como bases. Sus valores pueden estar almacenados en caché por Excel y no reflejar cambios recientes de las hojas maestras hasta que Excel recalcule.

La columna `Cuenta` del Excel es la identidad canónica del resultado. La columna `Cliente` puede representar una apertura. La relación entre ambas se refleja en `sites_proyecciones`.

Conciliación 2026

La alineación vigente se valida por cuenta, mes y componente contra las hojas calculadas del libro de referencia. No alcanza con comparar el total anual.

Ejecutar el control luego de cada carga o cambio de reglas:

```
python scripts\verificar_alineacion_proyectados.py "C:\ruta\Proyeccion Horas.xlsx" --year
2026
```

El comando devuelve error si encuentra una diferencia inesperada en `Facturacion Horas`, `Variable`, `Tarifacion` o `Next Gen`. Las celdas de salida que no tienen fórmula en el Excel se informan aparte como `FORMULA_FALTANTE`; con `--strict` también hacen fallar el control. La comparación monetaria se hace al centavo, pero los cálculos internos conservan precisión decimal completa.

La conciliación no reemplaza las reglas del negocio:

1. primero se corrigen horas, precios, porcentajes, sumas fijas y nombres;
2. después se calcula la diferencia residual;
3. cualquier residual se informa para corregir su componente de origen, sin generar ajustes artificiales;
4. el ajuste queda visible, auditable y reversible.

Si se cambia agosto en adelante:

1. actualizar el dato maestro en la app;
2. realizar el mismo cambio en el Excel si seguirá siendo la referencia;
3. recalcular el libro;
4. comparar nuevamente por cuenta y mes;
5. reemplazar la conciliación anterior desde el mes modificado.

Contratos de las APIs de Proyectados

Todas requieren sesión. Los métodos de escritura requieren CSRF y permisos.

Endpoint	Métodos	Responsabilidad
`/api/matriz-proyecciones`	GET, POST	Consultar y guardar proyecciones
`/api/campanias`	GET	Consultar IDs canónicos por cliente + campaña
`/api/matriz-proyecciones/<id>`	DELETE	Eliminar proyección
`/api/matriz-proyecciones/importar`	POST	Importar matriz
`/api/matriz-proyecciones/template`	GET	Descargar plantilla
`/api/proyecciones-plp/importar`	POST	Importar PLP anual
`/api/proyecciones-plp/template`	GET	Plantilla PLP
`/api/personal-distribucion`	GET, POST	Porcentajes diurnos/nocturnos
`/api/matriz-precios`	GET, POST	Consultar y guardar precios

Endpoint	Métodos	Responsabilidad
`/api/matriz-precios/importar`	POST	Importar precios
`/api/matriz-precios/inflacion`	POST	Aplicar índice
`/api/suma-fija`	GET, POST	Consultar y editar importes fijos
`/api/resumen-proyeccion`	GET	Facturación horas consolidada
`/api/sites-proyeccion`	POST	Regularizar site/cliente/campaña
`/api/variables`	GET, POST	Porcentajes y estimaciones
`/api/variables/importar`	POST	Importar variables
`/api/tarifaciones`	GET, POST	Montos adicionales
`/api/tarifaciones/importar`	POST	Importar tarifaciones
`/api/next-gen`	GET, POST	Dólar y productos
`/api/next-gen/importar`	POST	Importar Next Gen
`/api/control-proyecciones`	GET	Control de horas y dotaciones
`/api/calendario-operativo`	GET, POST	Feriados

Ejemplo de escritura de Suma fija:

```
POST /api/suma-fija
{
  "id": 123,
  "monto": 3750000
}
```

El `id` corresponde al registro de `matriz_precios` del mes.

Ejemplo conceptual de regularización:

```
POST /api/sites-proyeccion
{
  "site": "Mariano Quesada",
  "cliente_destino": "Santander Getnet",
  "campania_destino": "Santander Getnet",
  "origenes": [
    {
      "cliente": "Santander Getnet",
      "campania": "Santander Getnet Supervisor exclusivo"
```

```
    }
  ]
}
```

Seguridad y permisos para desarrolladores

Las escrituras desde navegador deben incluir:

```
Cookie de sesión válida
X-CSRF-Token o csrf_token válido
Origin/Referer permitido
Rol con permiso de edición
```

No se debe desactivar CSRF para resolver un `403`. Debe verificarse:

1. que la sesión no haya vencido;
2. que el `<meta name="csrf-token">` esté presente;
3. que `fetch` use `credentials: "same-origin"`;
4. que envíe `X-CSRF-Token`;
5. que el host utilizado esté incluido en `CORS_ORIGINS`.

En producción:

- configurar `APP_ENV=production`;
- usar un `SECRET_KEY` persistente y seguro;
- activar `SESSION_COOKIE_SECURE=1`;
- publicar únicamente detrás de HTTPS;
- usar PostgreSQL administrado;
- no exponer el servidor de desarrollo;
- respaldar la base antes de importaciones masivas.

Compartir en una red local

Para uso temporal dentro de una red privada:

```
$env:FLASK_HOST="0.0.0.0"
$env:PORT="8009"
python run.py
```

La otra persona accede mediante:

```
http://IP-PRIVADA-DEL-EQUIPO:8009
```

Esto no es una publicación segura en Internet. El equipo servidor debe permanecer encendido y ambos usuarios verán la misma base.

Backups y recuperación

Para SQLite, detener la aplicación o garantizar que no haya escrituras y copiar:

```
instance/facturacion.db
```

No reemplazar una base mientras Flask está escribiendo. Para restaurar:

1. detener Flask;
2. conservar una copia de la base actual;
3. reemplazar `facturacion.db`;
4. iniciar Flask;
5. revisar login, Historial y Facturación horas.

La carpeta `instance` puede contener respaldos operativos locales. No deben publicarse en repositorios si contienen información real.

Guía para extender un módulo

Al agregar una nueva funcionalidad persistente:

1. crear o ampliar el modelo en `app/models.py`;
2. agregar compatibilidad de esquema en `ensure_schema()` si no hay migración;
3. crear rutas HTML y API en `app/routes.py`;
4. validar datos en backend, no sólo en JavaScript;
5. proteger escrituras con sesión, rol y CSRF;
6. registrar snapshots en Historial;
7. habilitar deshacer cuando el cambio sea reversible;
8. agregar la pantalla Jinja correspondiente;
9. conectar el concepto al pipeline de cálculo;
10. documentar tabla, fórmula, API, importador y procedimiento operativo;
11. verificar que no se duplique al consolidar;
12. probar SQLite y, si aplica, PostgreSQL.

Checklist de verificación después de un cambio

```
[ ] La aplicación inicia sin errores.
[ ] Login responde y mantiene sesión.
[ ] Los GET no modifican datos.
[ ] Los POST/PUT/PATCH/DELETE exigen CSRF.
[ ] La pantalla guarda y vuelve a leer el mismo valor.
[ ] Historial muestra módulo, entidad y detalle correctos.
[ ] Deshacer restaura el snapshot anterior.
[ ] Facturación horas no duplica cuentas o conceptos.
[ ] Los doce meses suman el total anual.
[ ] Mayúsculas, minúsculas, acentos y Ñ no rompen el matching.
[ ] PLP conserva sus cuatro clasificaciones.
[ ] Suma fija se aplica una vez por mes.
[ ] Variable usa el porcentaje de la apertura correcta.
[ ] Next Gen usa el dólar del mismo mes.
[ ] No quedaron __pycache__, logs ni archivos temporales.
```

Diagnóstico de desvíos

Orden recomendado:

1. comparar el total por mes;
2. localizar la cuenta con diferencia;
3. separar Horas, Variable, Fijo mensual, Tarifación y Next Gen;
4. revisar nombres originales y destino en `sites_proyecciones`;
5. comparar horas proyectadas contra `matriz_proyecciones`;
6. verificar `precio_final`;
7. revisar porcentaje variable;
8. comprobar `importe_fijo_mensual`;
9. sumar tarifaciones conservando el signo;
10. verificar cantidad USD y dólar;
11. revisar el ajuste de conciliación sólo al final.

Una diferencia exactamente igual todos los meses suele indicar un fijo o una tarifación faltante. Una diferencia proporcional a las horas suele indicar un precio incorrecto. Una diferencia proporcional a Facturación horas suele indicar el porcentaje variable. Una diferencia que aparece sólo después de cierto mes suele indicar vigencia, inflación, cambio de campaña o una apertura nueva.

Anexo DDL completo - SQLite

Definición física generada desde `app/models.py`.

```
-- Dashboard de Facturación
-- Esquema físico completo para SQLITE
-- Generado desde app/models.py. No editar manualmente.

-- ============================================================
-- TABLA: campanias
-- ============================================================
CREATE TABLE campanias (
        id INTEGER NOT NULL,
        cliente VARCHAR(100) NOT NULL,
        nombre VARCHAR(100) NOT NULL,
        activa BOOLEAN NOT NULL,
        valor_hora_variable BOOLEAN,
        creado_en DATETIME NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_campanias_cliente_nombre UNIQUE (cliente, nombre)
);

-- ============================================================
-- TABLA: dashboard_operativo
-- ============================================================
CREATE TABLE dashboard_operativo (
        id INTEGER NOT NULL,
        year INTEGER NOT NULL,
        mes VARCHAR(7) NOT NULL,
        cliente VARCHAR(120) NOT NULL,
        campania VARCHAR(180) NOT NULL,
        datos TEXT NOT NULL,
        bajas_manual NUMERIC(20, 10) NOT NULL,
        creado_en DATETIME NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_dashboard_mes_cliente_campania UNIQUE (mes, cliente, campania)
);

CREATE INDEX ix_dashboard_operativo_cliente ON dashboard_operativo (cliente);

CREATE INDEX ix_dashboard_operativo_mes ON dashboard_operativo (mes);

CREATE INDEX ix_dashboard_operativo_year ON dashboard_operativo (year);

-- ============================================================
-- TABLA: dotaciones_clientes_mensuales
-- ============================================================
CREATE TABLE dotaciones_clientes_mensuales (
        id INTEGER NOT NULL,
```

```
        cliente VARCHAR(160) NOT NULL,
        fecha DATE NOT NULL,
        dotacion NUMERIC(20, 10) NOT NULL,
        creado_en DATETIME NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_dotacion_cliente_fecha UNIQUE (cliente, fecha)
);

CREATE INDEX ix_dotaciones_clientes_mensuales_cliente ON dotaciones_clientes_mensuales
(cliente);

CREATE INDEX ix_dotaciones_clientes_mensuales_fecha ON dotaciones_clientes_mensuales (fecha);

-- ============================================================
-- TABLA: dotaciones_mensuales
-- ============================================================
CREATE TABLE dotaciones_mensuales (
        id INTEGER NOT NULL,
        fecha DATE NOT NULL,
        dotacion_requerida NUMERIC(20, 10) NOT NULL,
        personal NUMERIC(20, 10) NOT NULL,
        creado_en DATETIME NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id)
);

CREATE UNIQUE INDEX ix_dotaciones_mensuales_fecha ON dotaciones_mensuales (fecha);

-- ============================================================
-- TABLA: excepciones_calculo
-- ============================================================
CREATE TABLE excepciones_calculo (
        id INTEGER NOT NULL,
        cliente VARCHAR(100) NOT NULL,
        campania VARCHAR(100) NOT NULL,
        tipo_calculo VARCHAR(60) NOT NULL,
        ajuste_vh_objetivo_pct NUMERIC(9, 4) NOT NULL,
        ajuste_vh_alcanzado_pct NUMERIC(9, 4) NOT NULL,
        activa BOOLEAN NOT NULL,
        creado_en DATETIME NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_excepcion_calculo_cliente_campania UNIQUE (cliente, campania)
);

-- ============================================================
-- TABLA: facturacion_anio
-- ============================================================
CREATE TABLE facturacion_anio (
        id INTEGER NOT NULL,
        fecha DATE NOT NULL,
        mes VARCHAR(20) NOT NULL,
        cliente VARCHAR(100) NOT NULL,
        gerente VARCHAR(100),
```

```
        jefe_site VARCHAR(100),
        campania VARCHAR(100),
        subcampania VARCHAR(100),
        tipo_negocio VARCHAR(100),
        es_next_gen BOOLEAN NOT NULL,
        objetivo_separar_ajuste_vh BOOLEAN NOT NULL,
        tipo_jornada VARCHAR(50) NOT NULL,
        horas_objetivo NUMERIC(20, 10) NOT NULL,
        horas_facturadas NUMERIC(20, 10) NOT NULL,
        horas_penalizadas NUMERIC(20, 10),
        valor_hora_objetivo NUMERIC(18, 2),
        valor_hora NUMERIC(18, 2) NOT NULL,
        facturado_horas_manual NUMERIC(18, 2),
        total_facturado_manual NUMERIC(18, 2),
        control_facturado_horas NUMERIC(18, 2),
        control_variable_productivo NUMERIC(18, 2),
        control_penalizaciones_bonos NUMERIC(18, 2),
        control_total_facturado NUMERIC(18, 2),
        control_objetivo_total NUMERIC(18, 2),
        tarifacion NUMERIC(18, 2),
        importe_fijo NUMERIC(18, 2),
        variable_objetivo NUMERIC(18, 2),
        variable_productivo NUMERIC(18, 2),
        bonos NUMERIC(18, 2),
        penalizaciones NUMERIC(18, 2),
        netx_gen NUMERIC(18, 2),
        otros NUMERIC(18, 2),
        PRIMARY KEY (id)
);

-- ============================================================
-- TABLA: feriados_operativos
-- ============================================================
CREATE TABLE feriados_operativos (
        id INTEGER NOT NULL,
        year INTEGER NOT NULL,
        fecha DATE NOT NULL,
        nombre VARCHAR(160) NOT NULL,
        tipo VARCHAR(30) NOT NULL,
        activo BOOLEAN NOT NULL,
        creado_en DATETIME NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_feriados_operativos_fecha UNIQUE (fecha)
);

CREATE INDEX ix_feriados_operativos_fecha ON feriados_operativos (fecha);

CREATE INDEX ix_feriados_operativos_year ON feriados_operativos (year);

-- ============================================================
-- TABLA: graficos_dotaciones_mensuales
-- ============================================================
CREATE TABLE graficos_dotaciones_mensuales (
```

```
        id INTEGER NOT NULL,
        fecha DATE NOT NULL,
        dotacion_requerida NUMERIC(20, 10) NOT NULL,
        activa_sl NUMERIC(20, 10) NOT NULL,
        activa_ba NUMERIC(20, 10) NOT NULL,
        creado_en DATETIME NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id)
);

CREATE UNIQUE INDEX ix_graficos_dotaciones_mensuales_fecha ON graficos_dotaciones_mensuales
(fecha);

-- ============================================================
-- TABLA: historico_clientes_mensuales
-- ============================================================
CREATE TABLE historico_clientes_mensuales (
        id INTEGER NOT NULL,
        year INTEGER NOT NULL,
        mes VARCHAR(7) NOT NULL,
        cliente VARCHAR(160) NOT NULL,
        datos_base TEXT NOT NULL,
        pagadas NUMERIC(20, 10),
        logueo NUMERIC(20, 10),
        creado_en DATETIME NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_historico_mes_cliente UNIQUE (mes, cliente)
);

CREATE INDEX ix_historico_clientes_mensuales_cliente ON historico_clientes_mensuales
(cliente);

CREATE INDEX ix_historico_clientes_mensuales_mes ON historico_clientes_mensuales (mes);

CREATE INDEX ix_historico_clientes_mensuales_year ON historico_clientes_mensuales (year);

-- ============================================================
-- TABLA: matriz_precios
-- ============================================================
CREATE TABLE matriz_precios (
        id INTEGER NOT NULL,
        site VARCHAR(100),
        cliente VARCHAR(100) NOT NULL,
        campania VARCHAR(100) NOT NULL,
        year INTEGER NOT NULL,
        mes VARCHAR(7) NOT NULL,
        precio_base NUMERIC(24, 10) NOT NULL,
        alcance_porcentaje NUMERIC(15, 10) NOT NULL,
        precio_final NUMERIC(24, 10) NOT NULL,
        importe_fijo_mensual NUMERIC(18, 2) NOT NULL,
        creado_en DATETIME NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id),
```

```
        CONSTRAINT uq_matriz_precios_cliente_campania_mes UNIQUE (cliente, campania, mes)
);

CREATE INDEX ix_matriz_precios_mes ON matriz_precios (mes);

CREATE INDEX ix_matriz_precios_year ON matriz_precios (year);

-- ============================================================
-- TABLA: matriz_proyecciones
-- ============================================================
CREATE TABLE matriz_proyecciones (
        id INTEGER NOT NULL,
        cliente VARCHAR(100) NOT NULL,
        campania VARCHAR(100) NOT NULL,
        year INTEGER NOT NULL,
        mes VARCHAR(7) NOT NULL,
        dotacion_requerida NUMERIC(20, 10) NOT NULL,
        carga_semanal VARCHAR(20) NOT NULL,
        carga_horaria NUMERIC(20, 10) NOT NULL,
        dias_objetivo INTEGER NOT NULL,
        horas_requeridas NUMERIC(20, 10) NOT NULL,
        porcentaje_cumplimiento NUMERIC(15, 10) NOT NULL,
        tiene_nocturnidad BOOLEAN NOT NULL,
        porcentaje_nocturnidad NUMERIC(15, 10) NOT NULL,
        tipo_plp VARCHAR(30),
        horas_carga_manual BOOLEAN NOT NULL,
        dias_objetivo_manual INTEGER,
        horas_requeridas_manual NUMERIC(20, 10),
        creado_en DATETIME NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_matriz_proyecciones_cliente_campania_mes UNIQUE (cliente, campania,
mes)
);

CREATE INDEX ix_matriz_proyecciones_mes ON matriz_proyecciones (mes);

CREATE INDEX ix_matriz_proyecciones_year ON matriz_proyecciones (year);

-- ============================================================
-- TABLA: next_gen_dolar
-- ============================================================
CREATE TABLE next_gen_dolar (
        id INTEGER NOT NULL,
        year INTEGER NOT NULL,
        mes VARCHAR(7) NOT NULL,
        valor NUMERIC(24, 10) NOT NULL,
        creado_en DATETIME NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id)
);

CREATE UNIQUE INDEX ix_next_gen_dolar_mes ON next_gen_dolar (mes);
```

```
CREATE INDEX ix_next_gen_dolar_year ON next_gen_dolar (year);

-- ============================================================
-- TABLA: next_gen_productos
-- ============================================================
CREATE TABLE next_gen_productos (
        id INTEGER NOT NULL,
        site VARCHAR(100),
        cliente VARCHAR(100) NOT NULL,
        campania VARCHAR(100) NOT NULL,
        producto VARCHAR(160) NOT NULL,
        year INTEGER NOT NULL,
        mes VARCHAR(7) NOT NULL,
        cantidad_usd NUMERIC(24, 10) NOT NULL,
        cotizacion_aplicada NUMERIC(24, 10),
        creado_en DATETIME NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_next_gen_producto_mes UNIQUE (cliente, campania, producto, mes)
);

CREATE INDEX ix_next_gen_productos_mes ON next_gen_productos (mes);

CREATE INDEX ix_next_gen_productos_year ON next_gen_productos (year);

-- ============================================================
-- TABLA: personal_distribucion_horas
-- ============================================================
CREATE TABLE personal_distribucion_horas (
        id INTEGER NOT NULL,
        servicio VARCHAR(100) NOT NULL,
        year INTEGER NOT NULL,
        mes VARCHAR(7) NOT NULL,
        porcentaje_diurno NUMERIC(15, 10) NOT NULL,
        creado_en DATETIME NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_personal_distribucion_servicio_mes UNIQUE (servicio, mes)
);

CREATE INDEX ix_personal_distribucion_horas_mes ON personal_distribucion_horas (mes);

CREATE INDEX ix_personal_distribucion_horas_servicio ON personal_distribucion_horas
(servicio);

CREATE INDEX ix_personal_distribucion_horas_year ON personal_distribucion_horas (year);

-- ============================================================
-- TABLA: proforma_personal
-- ============================================================
CREATE TABLE proforma_personal (
        id INTEGER NOT NULL,
        tipo_proforma VARCHAR(30) NOT NULL,
        estado_proforma VARCHAR(20) NOT NULL,
```

```
        fdv VARCHAR(150) NOT NULL,
        periodo VARCHAR(6) NOT NULL,
        negocio VARCHAR(100) NOT NULL,
        sitio_proveedor VARCHAR(100) NOT NULL,
        segmento VARCHAR(150) NOT NULL,
        subsitio VARCHAR(150),
        tipo_hora VARCHAR(50) NOT NULL,
        total_horas NUMERIC(20, 10) NOT NULL,
        precio NUMERIC(20, 10) NOT NULL,
        monto_fijo NUMERIC(20, 10) NOT NULL,
        porcentaje_bono_kpi_vs NUMERIC(15, 10) NOT NULL,
        monto_variable_kpi_vs NUMERIC(20, 10) NOT NULL,
        porcentaje_bono_ac NUMERIC(15, 10) NOT NULL,
        monto_variable_ac NUMERIC(20, 10) NOT NULL,
        monto_variable NUMERIC(20, 10) NOT NULL,
        total_proyeccion NUMERIC(20, 10) NOT NULL,
        bono_porcentaje_total NUMERIC(15, 10) NOT NULL,
        archivo_origen VARCHAR(255),
        creado_en DATETIME NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_proforma_personal_fila_tipo UNIQUE (tipo_proforma, fdv, periodo,
negocio, sitio_proveedor, segmento, subsitio, tipo_hora)
);

CREATE INDEX ix_proforma_personal_estado_proforma ON proforma_personal (estado_proforma);

CREATE INDEX ix_proforma_personal_periodo ON proforma_personal (periodo);

CREATE INDEX ix_proforma_personal_tipo_proforma ON proforma_personal (tipo_proforma);

-- ============================================================
-- TABLA: ratio_eli_ii_mensual
-- ============================================================
CREATE TABLE ratio_eli_ii_mensual (
        id INTEGER NOT NULL,
        fecha DATE NOT NULL,
        operaciones NUMERIC(20, 10) NOT NULL,
        staff NUMERIC(20, 10) NOT NULL,
        operaciones_importe NUMERIC(18, 2) NOT NULL,
        staff_importe NUMERIC(18, 2) NOT NULL,
        creado_en DATETIME NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id)
);

CREATE UNIQUE INDEX ix_ratio_eli_ii_mensual_fecha ON ratio_eli_ii_mensual (fecha);

-- ============================================================
-- TABLA: ratio_eli_mensual
-- ============================================================
CREATE TABLE ratio_eli_mensual (
        id INTEGER NOT NULL,
        fecha DATE NOT NULL,
```

```
        requerido NUMERIC(20, 10) NOT NULL,
        staff NUMERIC(20, 10) NOT NULL,
        creado_en DATETIME NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id)
);

CREATE UNIQUE INDEX ix_ratio_eli_mensual_fecha ON ratio_eli_mensual (fecha);

-- ============================================================
-- TABLA: sites_proyecciones
-- ============================================================
CREATE TABLE sites_proyecciones (
        id INTEGER NOT NULL,
        cliente VARCHAR(100) NOT NULL,
        campania VARCHAR(160) NOT NULL,
        site VARCHAR(100) NOT NULL,
        cliente_destino VARCHAR(100),
        campania_destino VARCHAR(160),
        creado_en DATETIME NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_site_proyeccion_cliente_campania UNIQUE (cliente, campania)
);

-- ============================================================
-- TABLA: tarifaciones_campanias
-- ============================================================
CREATE TABLE tarifaciones_campanias (
        id INTEGER NOT NULL,
        site VARCHAR(100),
        cliente VARCHAR(100) NOT NULL,
        campania VARCHAR(100) NOT NULL,
        concepto VARCHAR(100) NOT NULL,
        year INTEGER NOT NULL,
        mes VARCHAR(7) NOT NULL,
        monto NUMERIC(24, 10) NOT NULL,
        creado_en DATETIME NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_tarifacion_campania_concepto_mes UNIQUE (cliente, campania, concepto,
mes)
);

CREATE INDEX ix_tarifaciones_campanias_mes ON tarifaciones_campanias (mes);

CREATE INDEX ix_tarifaciones_campanias_year ON tarifaciones_campanias (year);

-- ============================================================
-- TABLA: usuarios
-- ============================================================
CREATE TABLE usuarios (
        id INTEGER NOT NULL,
        nombre VARCHAR(120) NOT NULL,
```

```
        email VARCHAR(160) NOT NULL,
        password_hash VARCHAR(255) NOT NULL,
        rol VARCHAR(30) NOT NULL,
        puesto VARCHAR(100),
        gerente_asignado VARCHAR(100),
        jefe_site_asignado VARCHAR(100),
        permisos_personalizados TEXT,
        activo BOOLEAN NOT NULL,
        debe_cambiar_password BOOLEAN NOT NULL,
        creado_en DATETIME NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id)
);

CREATE UNIQUE INDEX ix_usuarios_email ON usuarios (email);

-- ============================================================
-- TABLA: variables_campanias
-- ============================================================
CREATE TABLE variables_campanias (
        id INTEGER NOT NULL,
        site VARCHAR(100),
        cliente VARCHAR(100) NOT NULL,
        campania VARCHAR(100) NOT NULL,
        year INTEGER NOT NULL,
        mes VARCHAR(7) NOT NULL,
        porcentaje NUMERIC(15, 10) NOT NULL,
        creado_en DATETIME NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_variables_cliente_campania_mes UNIQUE (cliente, campania, mes)
);

CREATE INDEX ix_variables_campanias_mes ON variables_campanias (mes);

CREATE INDEX ix_variables_campanias_year ON variables_campanias (year);

-- ============================================================
-- TABLA: asignaciones_comerciales
-- ============================================================
CREATE TABLE asignaciones_comerciales (
        id INTEGER NOT NULL,
        campania_id INTEGER NOT NULL,
        cliente VARCHAR(100) NOT NULL,
        gerente VARCHAR(100) NOT NULL,
        jefe_site VARCHAR(100) NOT NULL,
        campania VARCHAR(100) NOT NULL,
        subcampania VARCHAR(100) NOT NULL,
        tipo_negocio VARCHAR(100),
        segmento VARCHAR(100),
        grupo_facturacion VARCHAR(100),
        es_next_gen BOOLEAN NOT NULL,
        activa BOOLEAN NOT NULL,
        vigencia_desde VARCHAR(7),
```

```
        vigencia_hasta VARCHAR(7),
        creado_en DATETIME NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT fk_asignaciones_campania FOREIGN KEY(campania_id) REFERENCES campanias
(id)
);

CREATE INDEX ix_asignaciones_comerciales_campania_id ON asignaciones_comerciales
(campania_id);

CREATE INDEX ix_asignaciones_comerciales_grupo_facturacion ON asignaciones_comerciales
(grupo_facturacion);

CREATE INDEX ix_asignaciones_comerciales_segmento ON asignaciones_comerciales (segmento);

CREATE INDEX ix_asignaciones_comerciales_vigencia_desde ON asignaciones_comerciales
(vigencia_desde);

CREATE INDEX ix_asignaciones_comerciales_vigencia_hasta ON asignaciones_comerciales
(vigencia_hasta);

-- ============================================================
-- TABLA: historial_cambios
-- ============================================================
CREATE TABLE historial_cambios (
        id INTEGER NOT NULL,
        usuario_id INTEGER,
        usuario_nombre VARCHAR(120),
        usuario_email VARCHAR(160),
        accion VARCHAR(30) NOT NULL,
        entidad VARCHAR(80) NOT NULL,
        entidad_id VARCHAR(50),
        resumen VARCHAR(255) NOT NULL,
        detalle TEXT,
        antes TEXT,
        despues TEXT,
        creado_en DATETIME NOT NULL,
        PRIMARY KEY (id),
        FOREIGN KEY(usuario_id) REFERENCES usuarios (id)
);

CREATE INDEX ix_historial_cambios_creado_en ON historial_cambios (creado_en);

-- ============================================================
-- TABLA: justificaciones_ajustes
-- ============================================================
CREATE TABLE justificaciones_ajustes (
        id INTEGER NOT NULL,
        facturacion_id INTEGER NOT NULL,
        tipo VARCHAR(30) NOT NULL,
        cantidad NUMERIC(18, 4),
        precio NUMERIC(18, 2),
        importe NUMERIC(18, 2) NOT NULL,
        descripcion TEXT NOT NULL,
```

```
        creado_en DATETIME NOT NULL,
        PRIMARY KEY (id),
        FOREIGN KEY(facturacion_id) REFERENCES facturacion_anio (id)
);

-- ============================================================
-- TABLA: matriz_proyecciones_jornadas
-- ============================================================
CREATE TABLE matriz_proyecciones_jornadas (
        id INTEGER NOT NULL,
        proyeccion_id INTEGER NOT NULL,
        dotacion_requerida NUMERIC(20, 10) NOT NULL,
        carga_semanal VARCHAR(30) NOT NULL,
        carga_horaria NUMERIC(20, 10) NOT NULL,
        dias_objetivo INTEGER NOT NULL,
        horas_requeridas NUMERIC(20, 10) NOT NULL,
        PRIMARY KEY (id),
        FOREIGN KEY(proyeccion_id) REFERENCES matriz_proyecciones (id)
);

CREATE INDEX ix_matriz_proyecciones_jornadas_proyeccion_id ON matriz_proyecciones_jornadas
(proyeccion_id);

-- ============================================================
-- TABLA: normalizacion_nombres_proyectados
-- ============================================================
CREATE TABLE normalizacion_nombres_proyectados (
        id INTEGER NOT NULL,
        campania_id INTEGER NOT NULL,
        cliente_proyectado VARCHAR(160) NOT NULL,
        servicio_proyectado VARCHAR(160) NOT NULL,
        actualizado_en DATETIME NOT NULL,
        PRIMARY KEY (id),
        FOREIGN KEY(campania_id) REFERENCES campanias (id)
);

CREATE UNIQUE INDEX ix_normalizacion_nombres_proyectados_campania_id ON
normalizacion_nombres_proyectados (campania_id);

-- ============================================================
-- TABLA: seguimiento_personal_importaciones
-- ============================================================
CREATE TABLE seguimiento_personal_importaciones (
        id INTEGER NOT NULL,
        archivo VARCHAR(255) NOT NULL,
        periodo_referencia VARCHAR(100),
        contenido_hojas BLOB NOT NULL,
        contenido_xlsx BLOB NOT NULL,
        cantidad_hojas INTEGER NOT NULL,
        cantidad_filas INTEGER NOT NULL,
        cantidad_formulas INTEGER NOT NULL,
        advertencias INTEGER NOT NULL,
        usuario_id INTEGER,
        creado_en DATETIME NOT NULL,
```

```
        PRIMARY KEY (id),
        FOREIGN KEY(usuario_id) REFERENCES usuarios (id)
);

CREATE INDEX ix_seguimiento_personal_importaciones_creado_en ON
seguimiento_personal_importaciones (creado_en);
```

Anexo DDL completo - PostgreSQL

Definición física generada desde `app/models.py`.

```
-- Dashboard de Facturación
-- Esquema físico completo para POSTGRESQL
-- Generado desde app/models.py. No editar manualmente.

-- ============================================================
-- TABLA: campanias
-- ============================================================
CREATE TABLE campanias (
        id SERIAL NOT NULL,
        cliente VARCHAR(100) NOT NULL,
        nombre VARCHAR(100) NOT NULL,
        activa BOOLEAN NOT NULL,
        valor_hora_variable BOOLEAN,
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_campanias_cliente_nombre UNIQUE (cliente, nombre)
);

-- ============================================================
-- TABLA: dashboard_operativo
-- ============================================================
CREATE TABLE dashboard_operativo (
        id SERIAL NOT NULL,
        year INTEGER NOT NULL,
        mes VARCHAR(7) NOT NULL,
        cliente VARCHAR(120) NOT NULL,
        campania VARCHAR(180) NOT NULL,
        datos TEXT NOT NULL,
        bajas_manual NUMERIC(20, 10) NOT NULL,
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_dashboard_mes_cliente_campania UNIQUE (mes, cliente, campania)
);

CREATE INDEX ix_dashboard_operativo_cliente ON dashboard_operativo (cliente);
```

```
CREATE INDEX ix_dashboard_operativo_mes ON dashboard_operativo (mes);

CREATE INDEX ix_dashboard_operativo_year ON dashboard_operativo (year);

-- ============================================================
-- TABLA: dotaciones_clientes_mensuales
-- ============================================================
CREATE TABLE dotaciones_clientes_mensuales (
        id SERIAL NOT NULL,
        cliente VARCHAR(160) NOT NULL,
        fecha DATE NOT NULL,
        dotacion NUMERIC(20, 10) NOT NULL,
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_dotacion_cliente_fecha UNIQUE (cliente, fecha)
);

CREATE INDEX ix_dotaciones_clientes_mensuales_cliente ON dotaciones_clientes_mensuales
(cliente);

CREATE INDEX ix_dotaciones_clientes_mensuales_fecha ON dotaciones_clientes_mensuales (fecha);

-- ============================================================
-- TABLA: dotaciones_mensuales
-- ============================================================
CREATE TABLE dotaciones_mensuales (
        id SERIAL NOT NULL,
        fecha DATE NOT NULL,
        dotacion_requerida NUMERIC(20, 10) NOT NULL,
        personal NUMERIC(20, 10) NOT NULL,
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id)
);

CREATE UNIQUE INDEX ix_dotaciones_mensuales_fecha ON dotaciones_mensuales (fecha);

-- ============================================================
-- TABLA: excepciones_calculo
-- ============================================================
CREATE TABLE excepciones_calculo (
        id SERIAL NOT NULL,
        cliente VARCHAR(100) NOT NULL,
        campania VARCHAR(100) NOT NULL,
        tipo_calculo VARCHAR(60) NOT NULL,
        ajuste_vh_objetivo_pct NUMERIC(9, 4) NOT NULL,
        ajuste_vh_alcanzado_pct NUMERIC(9, 4) NOT NULL,
        activa BOOLEAN NOT NULL,
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_excepcion_calculo_cliente_campania UNIQUE (cliente, campania)
);
```

```sql
-- ============================================================
-- TABLA: facturacion_anio
-- ============================================================
CREATE TABLE facturacion_anio (
        id SERIAL NOT NULL,
        fecha DATE NOT NULL,
        mes VARCHAR(20) NOT NULL,
        cliente VARCHAR(100) NOT NULL,
        gerente VARCHAR(100),
        jefe_site VARCHAR(100),
        campania VARCHAR(100),
        subcampania VARCHAR(100),
        tipo_negocio VARCHAR(100),
        es_next_gen BOOLEAN NOT NULL,
        objetivo_separar_ajuste_vh BOOLEAN NOT NULL,
        tipo_jornada VARCHAR(50) NOT NULL,
        horas_objetivo NUMERIC(20, 10) NOT NULL,
        horas_facturadas NUMERIC(20, 10) NOT NULL,
        horas_penalizadas NUMERIC(20, 10),
        valor_hora_objetivo NUMERIC(18, 2),
        valor_hora NUMERIC(18, 2) NOT NULL,
        facturado_horas_manual NUMERIC(18, 2),
        total_facturado_manual NUMERIC(18, 2),
        control_facturado_horas NUMERIC(18, 2),
        control_variable_productivo NUMERIC(18, 2),
        control_penalizaciones_bonos NUMERIC(18, 2),
        control_total_facturado NUMERIC(18, 2),
        control_objetivo_total NUMERIC(18, 2),
        tarifacion NUMERIC(18, 2),
        importe_fijo NUMERIC(18, 2),
        variable_objetivo NUMERIC(18, 2),
        variable_productivo NUMERIC(18, 2),
        bonos NUMERIC(18, 2),
        penalizaciones NUMERIC(18, 2),
        netx_gen NUMERIC(18, 2),
        otros NUMERIC(18, 2),
        PRIMARY KEY (id)
);

-- ============================================================
-- TABLA: feriados_operativos
-- ============================================================
CREATE TABLE feriados_operativos (
        id SERIAL NOT NULL,
        year INTEGER NOT NULL,
        fecha DATE NOT NULL,
        nombre VARCHAR(160) NOT NULL,
        tipo VARCHAR(30) NOT NULL,
        activo BOOLEAN NOT NULL,
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_feriados_operativos_fecha UNIQUE (fecha)
);
```

```
CREATE INDEX ix_feriados_operativos_fecha ON feriados_operativos (fecha);

CREATE INDEX ix_feriados_operativos_year ON feriados_operativos (year);

-- ============================================================
-- TABLA: graficos_dotaciones_mensuales
-- ============================================================
CREATE TABLE graficos_dotaciones_mensuales (
        id SERIAL NOT NULL,
        fecha DATE NOT NULL,
        dotacion_requerida NUMERIC(20, 10) NOT NULL,
        activa_sl NUMERIC(20, 10) NOT NULL,
        activa_ba NUMERIC(20, 10) NOT NULL,
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id)
);

CREATE UNIQUE INDEX ix_graficos_dotaciones_mensuales_fecha ON graficos_dotaciones_mensuales
(fecha);

-- ============================================================
-- TABLA: historico_clientes_mensuales
-- ============================================================
CREATE TABLE historico_clientes_mensuales (
        id SERIAL NOT NULL,
        year INTEGER NOT NULL,
        mes VARCHAR(7) NOT NULL,
        cliente VARCHAR(160) NOT NULL,
        datos_base TEXT NOT NULL,
        pagadas NUMERIC(20, 10),
        logueo NUMERIC(20, 10),
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_historico_mes_cliente UNIQUE (mes, cliente)
);

CREATE INDEX ix_historico_clientes_mensuales_cliente ON historico_clientes_mensuales
(cliente);

CREATE INDEX ix_historico_clientes_mensuales_mes ON historico_clientes_mensuales (mes);

CREATE INDEX ix_historico_clientes_mensuales_year ON historico_clientes_mensuales (year);

-- ============================================================
-- TABLA: matriz_precios
-- ============================================================
CREATE TABLE matriz_precios (
        id SERIAL NOT NULL,
        site VARCHAR(100),
        cliente VARCHAR(100) NOT NULL,
        campania VARCHAR(100) NOT NULL,
```

```
        year INTEGER NOT NULL,
        mes VARCHAR(7) NOT NULL,
        precio_base NUMERIC(24, 10) NOT NULL,
        alcance_porcentaje NUMERIC(15, 10) NOT NULL,
        precio_final NUMERIC(24, 10) NOT NULL,
        importe_fijo_mensual NUMERIC(18, 2) NOT NULL,
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_matriz_precios_cliente_campania_mes UNIQUE (cliente, campania, mes)
);

CREATE INDEX ix_matriz_precios_mes ON matriz_precios (mes);

CREATE INDEX ix_matriz_precios_year ON matriz_precios (year);

-- ============================================================
-- TABLA: matriz_proyecciones
-- ============================================================
CREATE TABLE matriz_proyecciones (
        id SERIAL NOT NULL,
        cliente VARCHAR(100) NOT NULL,
        campania VARCHAR(100) NOT NULL,
        year INTEGER NOT NULL,
        mes VARCHAR(7) NOT NULL,
        dotacion_requerida NUMERIC(20, 10) NOT NULL,
        carga_semanal VARCHAR(20) NOT NULL,
        carga_horaria NUMERIC(20, 10) NOT NULL,
        dias_objetivo INTEGER NOT NULL,
        horas_requeridas NUMERIC(20, 10) NOT NULL,
        porcentaje_cumplimiento NUMERIC(15, 10) NOT NULL,
        tiene_nocturnidad BOOLEAN NOT NULL,
        porcentaje_nocturnidad NUMERIC(15, 10) NOT NULL,
        tipo_plp VARCHAR(30),
        horas_carga_manual BOOLEAN NOT NULL,
        dias_objetivo_manual INTEGER,
        horas_requeridas_manual NUMERIC(20, 10),
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_matriz_proyecciones_cliente_campania_mes UNIQUE (cliente, campania,
mes)
);

CREATE INDEX ix_matriz_proyecciones_mes ON matriz_proyecciones (mes);

CREATE INDEX ix_matriz_proyecciones_year ON matriz_proyecciones (year);

-- ============================================================
-- TABLA: next_gen_dolar
-- ============================================================
CREATE TABLE next_gen_dolar (
        id SERIAL NOT NULL,
        year INTEGER NOT NULL,
```

```
        mes VARCHAR(7) NOT NULL,
        valor NUMERIC(24, 10) NOT NULL,
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id)
);

CREATE UNIQUE INDEX ix_next_gen_dolar_mes ON next_gen_dolar (mes);

CREATE INDEX ix_next_gen_dolar_year ON next_gen_dolar (year);

-- ============================================================
-- TABLA: next_gen_productos
-- ============================================================
CREATE TABLE next_gen_productos (
        id SERIAL NOT NULL,
        site VARCHAR(100),
        cliente VARCHAR(100) NOT NULL,
        campania VARCHAR(100) NOT NULL,
        producto VARCHAR(160) NOT NULL,
        year INTEGER NOT NULL,
        mes VARCHAR(7) NOT NULL,
        cantidad_usd NUMERIC(24, 10) NOT NULL,
        cotizacion_aplicada NUMERIC(24, 10),
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_next_gen_producto_mes UNIQUE (cliente, campania, producto, mes)
);

CREATE INDEX ix_next_gen_productos_mes ON next_gen_productos (mes);

CREATE INDEX ix_next_gen_productos_year ON next_gen_productos (year);

-- ============================================================
-- TABLA: personal_distribucion_horas
-- ============================================================
CREATE TABLE personal_distribucion_horas (
        id SERIAL NOT NULL,
        servicio VARCHAR(100) NOT NULL,
        year INTEGER NOT NULL,
        mes VARCHAR(7) NOT NULL,
        porcentaje_diurno NUMERIC(15, 10) NOT NULL,
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_personal_distribucion_servicio_mes UNIQUE (servicio, mes)
);

CREATE INDEX ix_personal_distribucion_horas_mes ON personal_distribucion_horas (mes);

CREATE INDEX ix_personal_distribucion_horas_servicio ON personal_distribucion_horas
(servicio);
```

```
CREATE INDEX ix_personal_distribucion_horas_year ON personal_distribucion_horas (year);

-- ============================================================
-- TABLA: proforma_personal
-- ============================================================
CREATE TABLE proforma_personal (
        id SERIAL NOT NULL,
        tipo_proforma VARCHAR(30) NOT NULL,
        estado_proforma VARCHAR(20) NOT NULL,
        fdv VARCHAR(150) NOT NULL,
        periodo VARCHAR(6) NOT NULL,
        negocio VARCHAR(100) NOT NULL,
        sitio_proveedor VARCHAR(100) NOT NULL,
        segmento VARCHAR(150) NOT NULL,
        subsitio VARCHAR(150),
        tipo_hora VARCHAR(50) NOT NULL,
        total_horas NUMERIC(20, 10) NOT NULL,
        precio NUMERIC(20, 10) NOT NULL,
        monto_fijo NUMERIC(20, 10) NOT NULL,
        porcentaje_bono_kpi_vs NUMERIC(15, 10) NOT NULL,
        monto_variable_kpi_vs NUMERIC(20, 10) NOT NULL,
        porcentaje_bono_ac NUMERIC(15, 10) NOT NULL,
        monto_variable_ac NUMERIC(20, 10) NOT NULL,
        monto_variable NUMERIC(20, 10) NOT NULL,
        total_proyeccion NUMERIC(20, 10) NOT NULL,
        bono_porcentaje_total NUMERIC(15, 10) NOT NULL,
        archivo_origen VARCHAR(255),
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_proforma_personal_fila_tipo UNIQUE (tipo_proforma, fdv, periodo,
negocio, sitio_proveedor, segmento, subsitio, tipo_hora)
);

CREATE INDEX ix_proforma_personal_estado_proforma ON proforma_personal (estado_proforma);

CREATE INDEX ix_proforma_personal_periodo ON proforma_personal (periodo);

CREATE INDEX ix_proforma_personal_tipo_proforma ON proforma_personal (tipo_proforma);

-- ============================================================
-- TABLA: ratio_eli_ii_mensual
-- ============================================================
CREATE TABLE ratio_eli_ii_mensual (
        id SERIAL NOT NULL,
        fecha DATE NOT NULL,
        operaciones NUMERIC(20, 10) NOT NULL,
        staff NUMERIC(20, 10) NOT NULL,
        operaciones_importe NUMERIC(18, 2) NOT NULL,
        staff_importe NUMERIC(18, 2) NOT NULL,
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id)
);
```

```
CREATE UNIQUE INDEX ix_ratio_eli_ii_mensual_fecha ON ratio_eli_ii_mensual (fecha);

-- ============================================================
-- TABLA: ratio_eli_mensual
-- ============================================================
CREATE TABLE ratio_eli_mensual (
        id SERIAL NOT NULL,
        fecha DATE NOT NULL,
        requerido NUMERIC(20, 10) NOT NULL,
        staff NUMERIC(20, 10) NOT NULL,
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id)
);

CREATE UNIQUE INDEX ix_ratio_eli_mensual_fecha ON ratio_eli_mensual (fecha);

-- ============================================================
-- TABLA: sites_proyecciones
-- ============================================================
CREATE TABLE sites_proyecciones (
        id SERIAL NOT NULL,
        cliente VARCHAR(100) NOT NULL,
        campania VARCHAR(160) NOT NULL,
        site VARCHAR(100) NOT NULL,
        cliente_destino VARCHAR(100),
        campania_destino VARCHAR(160),
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_site_proyeccion_cliente_campania UNIQUE (cliente, campania)
);

-- ============================================================
-- TABLA: tarifaciones_campanias
-- ============================================================
CREATE TABLE tarifaciones_campanias (
        id SERIAL NOT NULL,
        site VARCHAR(100),
        cliente VARCHAR(100) NOT NULL,
        campania VARCHAR(100) NOT NULL,
        concepto VARCHAR(100) NOT NULL,
        year INTEGER NOT NULL,
        mes VARCHAR(7) NOT NULL,
        monto NUMERIC(24, 10) NOT NULL,
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_tarifacion_campania_concepto_mes UNIQUE (cliente, campania, concepto,
mes)
);

CREATE INDEX ix_tarifaciones_campanias_mes ON tarifaciones_campanias (mes);
```

```
CREATE INDEX ix_tarifaciones_campanias_year ON tarifaciones_campanias (year);

-- ============================================================
-- TABLA: usuarios
-- ============================================================
CREATE TABLE usuarios (
        id SERIAL NOT NULL,
        nombre VARCHAR(120) NOT NULL,
        email VARCHAR(160) NOT NULL,
        password_hash VARCHAR(255) NOT NULL,
        rol VARCHAR(30) NOT NULL,
        puesto VARCHAR(100),
        gerente_asignado VARCHAR(100),
        jefe_site_asignado VARCHAR(100),
        permisos_personalizados TEXT,
        activo BOOLEAN NOT NULL,
        debe_cambiar_password BOOLEAN NOT NULL,
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id)
);

CREATE UNIQUE INDEX ix_usuarios_email ON usuarios (email);

-- ============================================================
-- TABLA: variables_campanias
-- ============================================================
CREATE TABLE variables_campanias (
        id SERIAL NOT NULL,
        site VARCHAR(100),
        cliente VARCHAR(100) NOT NULL,
        campania VARCHAR(100) NOT NULL,
        year INTEGER NOT NULL,
        mes VARCHAR(7) NOT NULL,
        porcentaje NUMERIC(15, 10) NOT NULL,
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT uq_variables_cliente_campania_mes UNIQUE (cliente, campania, mes)
);

CREATE INDEX ix_variables_campanias_mes ON variables_campanias (mes);

CREATE INDEX ix_variables_campanias_year ON variables_campanias (year);

-- ============================================================
-- TABLA: asignaciones_comerciales
-- ============================================================
CREATE TABLE asignaciones_comerciales (
        id SERIAL NOT NULL,
        campania_id INTEGER NOT NULL,
        cliente VARCHAR(100) NOT NULL,
        gerente VARCHAR(100) NOT NULL,
```

```
        jefe_site VARCHAR(100) NOT NULL,
        campania VARCHAR(100) NOT NULL,
        subcampania VARCHAR(100) NOT NULL,
        tipo_negocio VARCHAR(100),
        segmento VARCHAR(100),
        grupo_facturacion VARCHAR(100),
        es_next_gen BOOLEAN NOT NULL,
        activa BOOLEAN NOT NULL,
        vigencia_desde VARCHAR(7),
        vigencia_hasta VARCHAR(7),
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        CONSTRAINT fk_asignaciones_campania FOREIGN KEY(campania_id) REFERENCES campanias
(id)
);

CREATE INDEX ix_asignaciones_comerciales_campania_id ON asignaciones_comerciales
(campania_id);

CREATE INDEX ix_asignaciones_comerciales_grupo_facturacion ON asignaciones_comerciales
(grupo_facturacion);

CREATE INDEX ix_asignaciones_comerciales_segmento ON asignaciones_comerciales (segmento);

CREATE INDEX ix_asignaciones_comerciales_vigencia_desde ON asignaciones_comerciales
(vigencia_desde);

CREATE INDEX ix_asignaciones_comerciales_vigencia_hasta ON asignaciones_comerciales
(vigencia_hasta);

-- ============================================================
-- TABLA: historial_cambios
-- ============================================================
CREATE TABLE historial_cambios (
        id SERIAL NOT NULL,
        usuario_id INTEGER,
        usuario_nombre VARCHAR(120),
        usuario_email VARCHAR(160),
        accion VARCHAR(30) NOT NULL,
        entidad VARCHAR(80) NOT NULL,
        entidad_id VARCHAR(50),
        resumen VARCHAR(255) NOT NULL,
        detalle TEXT,
        antes TEXT,
        despues TEXT,
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        FOREIGN KEY(usuario_id) REFERENCES usuarios (id)
);

CREATE INDEX ix_historial_cambios_creado_en ON historial_cambios (creado_en);

-- ============================================================
-- TABLA: justificaciones_ajustes
```

```
-- ============================================================
CREATE TABLE justificaciones_ajustes (
        id SERIAL NOT NULL,
        facturacion_id INTEGER NOT NULL,
        tipo VARCHAR(30) NOT NULL,
        cantidad NUMERIC(18, 4),
        precio NUMERIC(18, 2),
        importe NUMERIC(18, 2) NOT NULL,
        descripcion TEXT NOT NULL,
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        FOREIGN KEY(facturacion_id) REFERENCES facturacion_anio (id)
);

-- ============================================================
-- TABLA: matriz_proyecciones_jornadas
-- ============================================================
CREATE TABLE matriz_proyecciones_jornadas (
        id SERIAL NOT NULL,
        proyeccion_id INTEGER NOT NULL,
        dotacion_requerida NUMERIC(20, 10) NOT NULL,
        carga_semanal VARCHAR(30) NOT NULL,
        carga_horaria NUMERIC(20, 10) NOT NULL,
        dias_objetivo INTEGER NOT NULL,
        horas_requeridas NUMERIC(20, 10) NOT NULL,
        PRIMARY KEY (id),
        FOREIGN KEY(proyeccion_id) REFERENCES matriz_proyecciones (id)
);

CREATE INDEX ix_matriz_proyecciones_jornadas_proyeccion_id ON matriz_proyecciones_jornadas
(proyeccion_id);

-- ============================================================
-- TABLA: normalizacion_nombres_proyectados
-- ============================================================
CREATE TABLE normalizacion_nombres_proyectados (
        id SERIAL NOT NULL,
        campania_id INTEGER NOT NULL,
        cliente_proyectado VARCHAR(160) NOT NULL,
        servicio_proyectado VARCHAR(160) NOT NULL,
        actualizado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        FOREIGN KEY(campania_id) REFERENCES campanias (id)
);

CREATE UNIQUE INDEX ix_normalizacion_nombres_proyectados_campania_id ON
normalizacion_nombres_proyectados (campania_id);

-- ============================================================
-- TABLA: seguimiento_personal_importaciones
-- ============================================================
CREATE TABLE seguimiento_personal_importaciones (
        id SERIAL NOT NULL,
        archivo VARCHAR(255) NOT NULL,
```

```
        periodo_referencia VARCHAR(100),
        contenido_hojas BYTEA NOT NULL,
        contenido_xlsx BYTEA NOT NULL,
        cantidad_hojas INTEGER NOT NULL,
        cantidad_filas INTEGER NOT NULL,
        cantidad_formulas INTEGER NOT NULL,
        advertencias INTEGER NOT NULL,
        usuario_id INTEGER,
        creado_en TIMESTAMP WITHOUT TIME ZONE NOT NULL,
        PRIMARY KEY (id),
        FOREIGN KEY(usuario_id) REFERENCES usuarios (id)
);

CREATE INDEX ix_seguimiento_personal_importaciones_creado_en ON
seguimiento_personal_importaciones (creado_en);
```